DYNARK: Making Groth16 Dynamic

Tianyu Zhang, Yupeng Ouyang, and Yupeng Zhang

University of Illinois Urbana Champaign
{tianyuz,oyjason,zhangyp}@illinois.edu

Abstract. In recent years, numerous new and more efficient constructions of zero-knowledge succinct non-interactive argument of knowledge (zkSNARK) have been proposed, motivated by their growing practical applications. However, in most schemes, when the witness is changed, the prover has to recompute the proof from scratch even if the new witness is close to the old one. This is inefficient for applications where proofs are generated for dynamically changing witnesses with small changes.

In this paper, we introduce DYNARK, a dynamic zkSNARK scheme that can update the proof in sublinear time when the change of the witness is small. DYNARK is built on top of the seminal zkSNARK protocol of Groth, 2016. In the semi-dynamic setting where new witnesses deviate from a single original witness, for an R1CS of size n, after a preprocessing of $O(n \log n)$ group operations on the original witness, it only takes $O(d)$ group operations and $O(d \log^2 d)$ field operations to update the proof for a new witness with distance d from the original witness, which is nearly optimal. In the fully-dynamic setting where witnesses are updated sequentially, the update time of DYNARK is $O(d\sqrt{n \log n})$ group operations and $O(d \log^2 d)$ field operations, where d is the distance between two consecutive witnesses. Both the proof size and the verifier time are $O(1)$, which are exactly the same as Groth16. Compared to the scheme in a prior work by Wang et al. 2024, we reduce the proof size from $O(\sqrt{n})$ to $O(1)$ without relying on pairing product arguments or another zkSNARK, and the update time and the verifier time of DYNARK are faster in practice.

Experimental results show that for $n = 2^{20}$, after a one-time preprocessing of 74.3 seconds, it merely takes 3 milliseconds to update the proof in our semi-dynamic zkSNARK for $d = 1$, and 60 milliseconds to update the proof in our fully-dynamic zkSNARK. These are 1433× and 73× faster than Groth16, respectively. The proof size is 192 bytes and the verifier time is 4.4 milliseconds. The system is fully compatible with any existing deployment of Groth16 without changing the trusted setup, the proof and the verification algorithm.

1 Introduction

A zero-knowledge succinct non-interactive argument of knowledge (zkSNARK) enables a prover to convince a verifier that an NP statement is true, without revealing any information about the secret witness. zkSNARKs have seen prominent developments in recent years and have found critical applications

in many domains. It has been used to improve the privacy and scalability of blockchains and cryptocurrencies [SCG+14, vit]. It has also been applied to prove machine learning computations [ZFZS20, LXZ21, WYX+21, GGJ+23, SLZ24, APKP24], photo authenticity [NT16, DCB25], internet data compliance [GAZ+22] and even the execution of any programs in zero-knowledge virtual machines (zkVMs) [RIS23]. Partly driven by these applications, the efficiency of zkSNARK schemes—especially prover time—has improved by orders of magnitude over the past decade.

However, a major limitation of almost all zkSNARKs is that they are static. To generate a new proof for an updated witness, the prover has to re-execute the proof generation algorithm, and the prover time is the same as generating the proof from scratch, even if only a single element of the witness is changed. This is undesirable in many applications where the witness in a new proof often differs only slightly from the old one. For example, in blockchain applications such as zkBridges [XZC+22] and zkRollups [vit], zkSNARK is used to prove the correctness of off-chain computations and proofs are verified by smart contracts on-chain. Some computations involve the balances of all accounts of a cryptocurrency, and only a small number of accounts changes when a new proof is generated. In machine learning applications such as proving the output of a large language model [SLZ24, QSL+25], the next token is generated by running the model on the prompt together with all previously produced tokens. The computations on the prompt and earlier tokens remain unchanged, and are reused for new tokens and new prompts.

To overcome this limitation, Wang et al. [WPSP25b] introduced the notion of *dynamic zkSNARKs*, where the time to update the proof for a new witness is sublinear in the size of the statement when the distance between the new witness and the old witness is small. They also proposed a new zkSNARK scheme that is dynamic. However, in the basic version of the protocol, the proof size is square-root in the size of the statement. The proof size can be further reduced by composing the zkSNARK with another zkSNARK scheme, but this leads to a high overhead on the concrete efficiency of the scheme. See Section 1.3 for a more detailed explanation.

1.1 Our Contributions

In this paper, we propose an efficient dynamic zkSNARK scheme, Dynark, directly on top of Groth16 [Gro16], one of the most widely used zkSNARKs in practice with extremely small proof size. In our scheme, the proof, the verification algorithm, and the prover key generated by the trusted setup remain exactly the same as Groth16. We view this as an important advantage of our scheme, as it is fully compatible with all existing adoptions of Groth16 in practice. Our main results are summarized below:

– First, we define and build a *semi-dynamic* zkSNARK scheme, where there is one fixed *original witness* and its proof. After a one-time preprocessing of complexity $O(n \log n)$ group operations on the original witness of size n, it

only takes $O(d)$ group operations and $O(d \log^2 d)$ field operations to generate a new proof, where d is the distance between the new witness and the original witness. Note that the number of group operations in our scheme is asymptotically optimal. The proof size and the verifier time are both $O(1)$, and are identical to those in Groth16 as mentioned above. Inspired by the techniques to build the semi-dynamic zkSNARK, we also propose a new algorithm to generate the Groth16 proof with 4 FFTs of size n, while the best known algorithms and implementations have at least 6 FFTs. This may be of independent interest.

We stress that semi-dynamic zkSNARKs are also useful in many applications where the updated witnesses always remain close to the original witness, regardless of the number of updates. For example, when proving computations of different fine-tuned machine learning models from an open-source large language model, the fine-tuned models are always close to the original model.

- Second, we lift our semi-dynamic zkSNARK to a *fully-dynamic* zkSNARK similar to that defined in [WPSP25b]. In this setting, the witness can keep changing and d denotes the distance between two consecutive versions of the witness. We obtain a fully-dynamic zkSNARK scheme with an update time of $O(d\sqrt{n \log n})$ group operations and $O(d \log^2 d)$ field operations, without any amortization. The proof size and the verifier time remain $O(1)$. Our approach provides a compiler to lift any semi-dynamic zkSNARK to a fully-dynamic zkSNARK as long as the semi-dynamic zkSNARK satisfies some additional properties. Our technique can be applied to other works such as the lookup argument in [DGP+24] to make them dynamic in the de-amortized setting.
- Last, we have fully implemented DYNARK and benchmarked its performance. For $n = 2^{20}$, with a one-time preprocessing, it merely takes 3 milliseconds to update a proof with $d = 1$ in our semi-dynamic zkSNARK, and 60 milliseconds in our fully-dynamic zkSNARK. Theses are 1433× and 73× faster than recomputing the proof from scratch in Groth16. The update time of the semi-dynamic zkSNARK is faster than Groth16 for distance up to $d = 2^{15}$, while the fully-dynamic zkSNARK achieves faster updates for distance up to $d = 2^8$. The code of our implementation is available at `https://github.com/OYJason08/DYNARK_`.

1.2 Technical Overview

As observed in [WPSP25b], it is very challenging to build dynamic zkSNARKs from schemes relying on the Fiat-Shamir transformation [FS87]. This is because the change of the witness leads to the change of the transcript that includes the commitment of the witness. Therefore, the randomness derived from the Fiat-Shamir transformation becomes completely different, and the prover has to re-execute the prover algorithm from scratch. This observation rules out many recent zkSNARKs, such as those based on multivariate sumchecks, interactive oracle proofs and error-correcting codes.

In the remaining candidate zkSNARKs based on univariate polynomials, the most challenging step to make dynamic is of the form

$$q(X) = \frac{A(X) \cdot B(X) - C(X)}{v_{\mathbb{H}}(X)},$$

where roughly speaking $A(X)$ is defined by the "left" input of multiplication gates, $B(X)$ is defined by the "right" input, $C(X)$ is defined by the output, $v_{\mathbb{H}}(X)$ is the vanishing polynomial and $q(X)$ is the quotient polynomial of the division (see Section 2 for formal definitions). The proof consists of the term $[q(\tau)]_1$, denoting the generator of a group raised to the power of $q(\tau)$, the evaluation of $q(X)$ at a secret point τ. This computation appears in Plonk [GWC19], Marlin [CHM+20], as well as the dynamic zkSNARK in [WPSP25b]. As for the Groth16 [Gro16], the computation is slightly different, as $A(X), B(X), C(X)$ are computed by the R1CS matrices, and the prover key and the proof are different, but the main structure of the computation remains the same. See Section 2.1 for the formal description of Groth16.

As the degree of $C(X)$ is less than $v_{\mathbb{H}}(X)$, it does not affect the quotient polynomial $q(X)$ and we remove it from the equation to simplify the explanation. When the witness changes, the polynomials become $A'(X) = A(X) + \Delta^A(X)$ and $B'(X) = B(X) + \Delta^B(X)$. We express the difference polynomials using the Lagrange polynomials $\ell_i(X)$ as $\Delta^A(X) = \sum_{i \in I_A} \Delta_i^A \ell_i(X)$ and $\Delta^B(X) = \sum_{i \in I_B} \Delta_i^B \ell_i(X)$ where I_A, I_B are indices of non-zero elements in A, B respectively. We assume $|I_A| = O(d), |I_B| = O(d)$ for $d << n$ when the update on the witness is small. To generate the new proof, the prover needs to compute $[q'(\tau)]_1$ where $q'(X)$ is the quotient polynomial of the polynomial division[1]:

$$\begin{aligned}\frac{A'(X) \cdot B'(X)}{v_{\mathbb{H}}(X)} &= \frac{(A(X) + \Delta^A(X)) \cdot (B(X) + \Delta^B(X))}{v_{\mathbb{H}}(X)} \\ &= \frac{A(X) \cdot B(X)}{v_{\mathbb{H}}(X)} + \frac{\Delta^A(X) \cdot B(X)}{v_{\mathbb{H}}(X)} + \frac{A(X) \cdot \Delta^B(X)}{v_{\mathbb{H}}(X)} + \frac{\Delta^A(X) \cdot \Delta^B(X)}{v_{\mathbb{H}}(X)}.\end{aligned}$$

The quotient polynomial of the first term is exactly $q(X)$ and $[q(\tau)]_1$ is already computed in the original proof. It remains challenging to compute the quotient polynomials of the last three terms in time proportional only to d.

Projection term. We first show how to efficiently compute the second and third terms, which we name as the projection terms. We observe that $\frac{\Delta^A(X) \cdot B(X)}{v_{\mathbb{H}}(X)}$ is of exactly the same format as the lookup argument proposed in cq [EFG22]. In particular,

$$\frac{\Delta^A(X) \cdot B(X)}{v_{\mathbb{H}}(X)} = \sum\nolimits_{i \in I_A} \Delta_i^A \frac{\ell_i(X) \cdot B(X)}{v_{\mathbb{H}}(X)}$$

and the group element $[\frac{\ell_i(\tau) \cdot B(\tau)}{v_{\mathbb{H}}(\tau)}]_1$ can be precomputed for all $i \in [n]$ in $O(n \log n)$ group operations using the algorithm in [FK23]. With the preprocessing, all the

[1] Here we abuse the notation to mean the quotient polynomial of polynomial division with a remainder polynomial that is not shown in the equation.

$[\frac{\Delta^A(\tau)\cdot B(\tau)}{v_{\mathbb{H}}(\tau)}]_1$ can be computed by $|I_A| = O(d)$ group operations. The same approach works for $\frac{A(X)\cdot\Delta^B(X)}{v_{\mathbb{H}}(X)}$.

The full protocol is more complicated and in Section 3.4, we show how to compute the projection terms in Groth16 without changing the prover key generated by the trusted setup. We also propose a better preprocessing algorithm that improves the running time by 2× compared to [FK23].

Cross term. The remaining challenge is the last term named as the cross term:

$$\frac{\Delta^A(X)\cdot\Delta^B(X)}{v_{\mathbb{H}}(X)} = \sum\nolimits_{i\in I_A, j\in I_B} \Delta_i^A\Delta_j^B \frac{\ell_i(X)\ell_j(X)}{v_{\mathbb{H}}(X)}.$$

Here we cannot precompute $[\frac{\ell_i(\tau)\ell_j(\tau)}{v_{\mathbb{H}}(\tau)}]_1$ for all i, j since it would incur an overhead of $O(n^2)$ group operations, which is not practical. Fortunately, we show that $\frac{\ell_i(X)\ell_j(X)}{v_{\mathbb{H}}(X)}$ itself is sparse. That is, when $i \neq j$, the polynomial is of degree $n-2$ and is non-zero only at i and j. As shown in Theorem 1, its closed-form is

$$\frac{\ell_i(X)\ell_j(X)}{v_{\mathbb{H}}(X)} = \frac{\omega^{j-i}}{n(1-\omega^{j-i})}\ell_i(X) + \frac{\omega^{i-j}}{n(1-\omega^{i-j})}\ell_j(X).$$

Therefore, with $[\ell_i(\tau)]_1$ in the public parameters for all $i \in [n]$, $[\frac{\ell_i(\tau)\ell_j(\tau)}{v_{\mathbb{H}}(\tau)}]_1$ can be computed in $O(1)$ time. To compute the cross term in the proof, note that the group elements with the same $[\ell_i(\tau)]_1$ can be grouped together, making the complexity $O(d)$ group operations. Computing the constants in the equation for all $i \in I_A, j \in I_B$ takes $O(d^2)$ field operations. This can further be reduced to $O(d\log^2 d)$ using an algorithm for sparse convolutions. In Section 3.3, we further show the approach to compute the terms with $i = j$ in a unified framework with the projection terms above. This technique can also be used to construct a *sparse zkSNARK* on top of Groth16 where the prover time is only $O(m\log^2 m)$ field operations and $O(m)$ group operations for a witness with m non-zero elements. We defer the full construction to the full version.

Combining the two protocols above, we can update the quotient polynomial in the proof efficiently. All other elements in the proof are linear and can be updated efficiently in $O(d)$ time in the first place. This gives us the semi-dynamic zkSNARK scheme.

From semi-dynamic to fully-dynamic. The reason why the scheme above is not fully dynamic is because of the preprocessing to compute $[\frac{\ell_i(\tau)\cdot B(\tau)}{v_{\mathbb{H}}(\tau)}]_1$ and $[\frac{A(\tau)\cdot\ell_i(\tau)}{v_{\mathbb{H}}(\tau)}]_1$ for $i \in [n]$, which depends on the original witness. The prover cannot rerun the preprocessing for every new witness, which would take $O(n\log n)$ time for every update. The prover could rerun the preprocessing only when the distance between the new witness and the original witness reaches $d = O(\sqrt{n\log n})$, making the update time $O(d\sqrt{n\log n})$ in the amortized setting. Instead of this approach, we propose a new algorithm that de-amortizes the cost and yields a fully-dynamic zkSNARK.

The algorithm is inspired by the dynamic algorithm for convolutions in [RT97]. Let $z = (1, \mathbb{x}, \mathbb{w})$ be the extended witness of an R1CS. For a sequence of extended

witnesses $(z_0, z_1, z_2, z_3 \ldots)$, we divide them into blocks $(B_0, B_1, B_2, \ldots)$ such that the distance between the first witness in a block and the first witness in its previous block is no more than b. We can safely assume that the distance of any single update is no more than $O(\sqrt{n \log n})$, as otherwise it would be more efficient to just rerun the prover algorithm from scratch to generate the proof in our scheme.

In our fully-dynamic construction, we initialize the system by running the semi-dynamic preprocessing once on z_0. For any witness in the first block B_0, we then reuse our semi-dynamic scheme (with preprocessing anchored at z_0) to update the proof, with the update time bounded by $O(b)$ group operations and $O(b \log^2 b)$ field operations. Starting from witnesses in B_1, the prover is going to perform the following two computations:

1. Generate the new proofs still using the preprocessed data of z_0 and our semi-dynamic zkSNARK.
2. Run the preprocessing for the first witness in B_1.

Since the distance within one block is bounded by b, the distance between witnesses in B_1 and z_0 is no more than $2b$, thus the first item to generate the proof can still be computed by $O(b)$ group operations and $O(b \log^2 b)$ field operations. For the second item, we further require that the preprocessing can be computed step-by-step "smoothly". Our preprocessing satisfies this property using the classical butterfly circuit for DFTs and convolutions, see Section 4.1 for the formal protocols. With this property, we evenly divide the computation of the preprocessing into $O(\frac{n \log n}{b})$ per unit distance, and assign $O(\frac{dn \log n}{b})$ of computation to each witness in B_1, where d is used to denote its distance from the previous witness. At the end of B_1, the preprocessing is completed for the first witness in B_1. The preprocessed data of the first witness in B_1 can then be used to generate the proofs for all witnesses in B_2.

The same process is repeated where for any block B_i, the prover has the preprocessed data for the first witness in B_{i-1} ready and generates the proofs using our semi-dynamic zkSNARK. At the same time, the prover runs the preprocessing for the first witness in B_i step by step. By setting $b = O(\sqrt{n \log n})$, we obtain a fully-dynamic zkSNARK scheme with the update time of $O(d\sqrt{n \log n})$ group operations and $O(\sqrt{n \log n} \log^2 n)$ field operations without any amortization. In Section 4.1, we further show how to improve the number of field operations to $O(d \log^2 d)$ utilizing the "locality" property of convolutions. Our approach provides a compiler to lift any semi-dynamic zkSNARK to a fully-dynamic zkSNARK as long as the preprocessing can be computed step by step.

1.3 Related Work

The most relevant prior work is by Wang et al. in [WPSP25b], which initiated the study of dynamic zkSNARKs. They proposed a new permutation argument with a constant proof size that can be updated in time $O(d)$. Unfortunately, in order to handle the multiplication gates, they utilized a bucketing technique that

scheme	keygen time	prover time	update time	proof size
[Gro16]	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
[WPSP25b] (Basic)	$O(n)$	$O(n \log n)$	$O(d\sqrt{n} \log n)$	$O(\sqrt{n})$
[WPSP25b] (Dynaverse)	$O(n)$	$O(n \log n)$	$O(d\sqrt{n} \log n)$	$O(\log n)$
[WPSP25b] (Dynalog)	$O(n)$	$O(n \log^2 n)$	$O(d \log^3 n)$	$O(\log n)$
Ours–Semi	$O(n \log n)$	$O(n \log n)$	$O(d)\mathbb{G} + O(d \log^2 d)\mathbb{F}$	$O(1)$
Ours–Fully	$O(n \log n)$	$O(n \log n)$	$O(d\sqrt{n \log n})\mathbb{G} + O(d \log^2 d)\mathbb{F}$	$O(1)$

Table 1: Comparison with Groth16 and [WPSP25b].

divides the witness into $\sqrt{n}$ buckets of size $\sqrt{n}$, leading to a proof size of $O(\sqrt{n})$. We call this version the basic version and Table 1 shows its comparison with our schemes. The major improvement of our scheme over [WPSP25b] is that the proof size is constant, and is exactly the same as Groth16 with full compatibility. Moreover, in the semi-dynamic setting, our update time is much faster and does not depend on n at all. As we show in Section 5, our prover time, proof size and verifier time are all better than [WPSP25b] in the experiments. The scheme in [WPSP25b] can work with a universal trusted setup, but the running time and the prover key size would become $O(n\sqrt{n})$. In the latest version, the authors proposed an approach to reduce them to $O(n)$, at the cost of relying on the Fiat-Shamir transformation in the random oracle model. The complexity in the table refers to the variant with a per-circuit trusted setup, which is in the same setting as our schemes. In fact, the techniques proposed in our paper can be applied to improve the protocol for checking multiplication gates in [WPSP25b] and reduce its proof size to $O(1)$.

The authors of [WPSP25b] also proposed several variants on top of the basic scheme with better asymptotics. However, all of them utilize GIPA [BMM+21] or another zkSNARK such as Groth16. For example, in one variant named Dynaverse, the proof size and the verifier time are reduced to $O(\log n)$ or $O(1)$ by checking the pairing equations of the basic version using GIPA or Groth16. In another variant named Dynalog, an hierarchical data structure is introduced to reduce the update time to $O(d \log^3 n)$, and the proof size and the verifier time to $O(\log n)$, while increasing the prover time to $O(n \log^2 n)$. Again, it uses Groth16 to prove the correctness of the data structure and the verification of a vector commitment. In these variants, the verification of the inner SNARK involving group operations needs to be implemented in the outer SNARK, incurring a high overhead in the concrete efficiency. They were not implemented in [WPSP25b] and the performance is not clear in practice.

Finally, as a folklore construction mentioned in [WPSP25b], one can build a dynamic zkSNARK using a recursive SNARK over a tree structure (proof-carrying-data). The recursive SNARK of this approach needs to instantiate the random oracle in the SNARK with heuristic security, and the concrete efficiency is not clear in practice either.

In addition to dynamic zkSNARKs, other related works have studied related notions of updatability. The authors of [GGK+25] proposed *split-prover* zkSNARKs, which allow a prover to delegate part of the proof computation to a second prover when the full witness is known later. Our techniques can be used to improve the efficiency of their second prover when the R1CS is sparse. Updatable lookup arguments were studied in [DGP+24] and updatable KZG vector commitments were studied in [TAB+20, LJGK25]. Notably, [LJGK25] re-casted the preprocessing of [EFG22] as Cauchy-matrix multiplications and obtained the same improvement as our quotient-decomposition approach to speed up batched updates.

Since the early constructions by Kilian [Kil92] and Micali [Mic00], there have been numerous efficient constructions of zkSNARKs in recent years based on various techniques. There are zkSNARKs based on bilinear pairing [PHGR13, Gro16, GWC19], discrete logarithm [BBB+18, BMM+21, BFS20], interactive proofs and sumcheck [WTS+18, XZZ+19, Set20, CBBZ22], interactive oracle proofs [BSCR+19, BSBHR18, ZCF24], error-correcting codes [AHIV17, GLS+23, XZS22, BFK+24] and lattices [BC24]. See [Tha22] for a survey on all recent zkSNARKs. Unfortunately, as explained above, it is very challenging to turn those zkSNARKs with the Fiat-Shamir transformation into dynamic ones. This leaves us with only a few candidates, and Groth16 stands out as one of the most widely used zkSNARKs with extremely small proof size.

Concurrent work. In a new version of [WPSP25c], the authors proposed a new scheme that is different from those in prior versions. The scheme has $O(1)$ proof size without relying on GIPA or Groth16. The construction is essentially a semi-dynamic zkSNARK as defined in our paper, and is made fully-dynamic in the amortized setting. Compared to this protocol, our schemes are more efficient in practice and are fully compatible with Groth16, while the scheme in [WPSP25c] only needs a universal trusted setup.

1.4 Open Problems

Our fully-dynamic zkSNARK has an update time that is square-root in the size of the witness. It remains open to construct a dynamic zkSNARK with a poly-logarithmic update time, without relying on recursive SNARKs. It is particularly challenging to improve our construction, because the preprocessing involves convolutions and there are known lower bounds (square-root in the size of vectors) on dynamic algorithms for convolutions [FHM01]. Dynamic zkSNARKs based on multivariate polynomials could be a potential direction, which is left as interesting future work.

2 Preliminary

Notations. We use λ to denote the security parameter. $\mathbb{F}$ denotes a finite field of prime order p. PPT stands for probabilistic polynomial time. $\mathsf{negl}(\lambda)$ denotes a negligible function in λ. Let $[n]$ denote the set $\{0, 1, \ldots, n-1\}$ and $[k, n]$ denote

the set $\{k, k+1, \ldots, n-1\}$. We use bold letters to denote vectors such as $\mathbf{a}$ or use parentheses notation, such as $(a_i)_{i=0}^{n-1} := (a_0, a_1, \ldots, a_{n-1})$ to explicitly express each element. When $\mathbf{a}$ is sparse, we use $|\mathbf{a}|$ to denote the number of non-zero elements in vector $\mathbf{a}$ and use $I_{\mathbf{a}}$ to denote the set of indices for these non-zero elements. We use lowercase letters to denote elementary polynomials such as $\ell(X)$, uppercase letters to denote synthetic polynomials such as $A(X) = \sum_{i\in[n]} a_i \ell_i(X)$. Let $\circ$ denote the component-wise vector multiplication. We use $\mathbb{H} = \{1, \omega, \omega^2, \ldots, \omega^{n-1}\}$ be the multiplicative subgroup of the finite field $\mathbb{F}$, $v_{\mathbb{H}}(X) := \prod_{i\in[n]}(X - \omega^i) = X^n - 1$ to denote the vanishing polynomials and $\ell_i(X) = \prod_{j\in[n]/i}\left(\frac{X-\omega^j}{\omega^i-\omega^j}\right)$ to denote the Lagrange polynomials. We use $\widehat{\mathbf{a}} = \mathrm{DFT}(\mathbf{a}) = F\cdot\mathbf{a}$ to denote the discrete Fourier transformation (DFT) of length n, where F is the Fourier matrix defined as $\left(\omega^{ij}\right)_{(i,j)\in[n]^2} \in \mathbb{F}^{n\times n}$. We use $\mathbf{a} * \mathbf{b} = \mathbf{c}$ to denote the circular convolution of size n, which is defined as $\mathbf{c}[i] = \sum_{j\in[n]} a[j]\cdot b[i-j \bmod n]$ for each $i \in [n]$.

Bilinear pairing. $(p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g, h) \leftarrow \mathsf{BilGen}(1^\lambda)$ denotes the generation of a bilinear map where $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ are groups of prime order p, and g, h are generators of $\mathbb{G}_1, \mathbb{G}_2$, respectively. $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ is an efficiently computable and non-degenerate pairing and $e(g, h)$ is a generator of $\mathbb{G}_T$. Following the notation of [Gro16], we write $\mathbb{G}_1$ and $\mathbb{G}_2$ additively. We write $[a]_1 := a \cdot g$ and $[a]_2 := a \cdot h$. With this notation, $g = [1]_1, h = [1]_2$. For all $[a]_1 \in \mathbb{G}_1, [b]_2 \in \mathbb{G}_2$, $[a]_1 \cdot [b]_2 = e(a, b) = [a \cdot b]_T$. We use $[\mathbf{a}]_1$ to denote the vector of group elements in $\mathbb{G}_1$ by computing $([a_i]_1)_{i=0}^{n-1}$. The same notation applies to $\mathbb{G}_2$ and $\mathbb{G}_T$.

2.1 zkSNARK

Indexed Relation. An *indexed relation* $\mathcal{R}$ is a set of triples $(\mathtt{i}, \mathtt{x}, \mathtt{w})$ where $\mathtt{i}$ is the index, $\mathtt{x}$ is the instance and $\mathtt{w}$ is the witness. The corresponding *indexed language* $\mathcal{L}(\mathcal{R})$ consists of all pairs $(\mathtt{i}, \mathtt{x})$ for which there exists a witness $\mathtt{w}$ such that $(\mathtt{i}, \mathtt{x}, \mathtt{w}) \in \mathcal{R}$. For example, the indexed relation of satisfiable arithmetic circuits consists of triples where $\mathtt{i}$ is the description of an arithmetic circuit, $\mathtt{x}$ is a partial assignment to its input wires, and $\mathtt{w}$ is an assignment to the remaining input wires that makes the circuit output 1.

The formal definitions of zkSNARKs, the algebraic group model (AGM) and the q-DLOG assumption are presented in the full version.

R1CS. *Rank-1 Constraint System (R1CS)* is an NP-complete language that generalizes arithmetic circuit satisfiability. The indexed relation $\mathcal{R}_{\mathrm{R1CS}}$ is the set of all triples $(\mathtt{i}, \mathtt{x}, \mathtt{w}) = ((\mathbb{F}, n, m, k, A, B, C), \mathtt{x}, \mathtt{w})$ where $\mathbb{F}$ is a finite field and $A, B, C \in \mathbb{F}^{n\times m}, \mathtt{x} \in \mathbb{F}^{k-1}, \mathtt{w} \in \mathbb{F}^{m-k}$. The relation holds if only if $(A \cdot z) \circ (B \cdot z) = C \cdot z$ where $z = (1, \mathtt{x}, \mathtt{w}) \in \mathbb{F}^m$ is called the extended witness. Sometimes we refer to z as the witness when it is clear from the context.

Groth16. Groth16 [Gro16] is a zkSNARK for R1CS and the scheme is presented in Protocol 1. In $Groth16.\mathcal{G}$, polynomials u, v, w are defined as $u_i(X) = \sum_{j\in[n]} A_{j,i}\ell_j(X), v_i(X) = \sum_{j\in[n]} B_{j,i}\ell_j(X), w_i(X) = \sum_{j\in[n]} C_{j,i}\ell_j(X) \subseteq \mathbb{F}^{<n}[X]$

Protocol 1 (Groth16 zk-SNARKs)

- $Groth16.\mathcal{G}(1^\lambda, \mathbb{i}) \to (\mathbf{pk}, \mathbf{vk})$
  1. *Sample* $\alpha, \beta, \gamma, \delta, \tau \leftarrow \mathbb{Z}_p^*$
  2. *Let* $\mathbf{u} = (u_i(\tau))_{i=0}^{m-1}, \mathbf{v} = (v_i(\tau))_{i=0}^{m-1}, \mathbf{w} = (w_i(\tau))_{i=0}^{m-1}$
  3. *Let* $\boldsymbol{\tau} = \left(\frac{\tau^i(\tau^n-1)}{\delta}\right)_{i=0}^{n-2}$
  4. *Let* $\mathbf{t} = (\mathbf{t}^{\mathsf{v}}, \mathbf{t}^{\mathsf{p}})$
     where $\mathbf{t}^{\mathsf{v}} = \left(\frac{\beta u_i(\tau)+\alpha v_i(\tau)+w_i(\tau)}{\gamma}\right)_{i=0}^{k-1}, \mathbf{t}^{\mathsf{p}} = \left(\frac{\beta u_i(\tau)+\alpha v_i(\tau)+w_i(\tau)}{\delta}\right)_{i=k}^{m-1}$
  5. *Output* $\mathbf{vk} = ([\alpha]_1, [\beta]_2, [\gamma]_2, [\delta]_2, [\mathbf{t}^{\mathsf{v}}]_2)$,
     $\mathbf{pk} = \left(\mathbf{vk}, [\beta]_1, [\delta]_1, \left\{[\tau^i]_2\right\}_{i=0}^{n-1}, [\boldsymbol{\tau}]_1, [\mathbf{t}^{\mathsf{p}}]_1, [\mathbf{u}]_1, [\mathbf{v}]_1, [\mathbf{v}]_2\right)$
- $Groth16.\mathcal{P}(\mathbf{pk}, \mathbb{x}, \mathbb{w}) \to \pi$
  1. *Sample* $r, s \leftarrow \mathbb{Z}_p$
  2. *Compute* $[Q]_1 = \left[\frac{q(\tau)(\tau^n-1)}{\delta}\right]_1$ *where* $q(X)$ *satisfying* $A(X) \cdot B(X) - C(X) = q(X) \cdot v_{\mathbb{H}}(X)$
  3. *Let* $z = (1, \mathbb{x}, \mathbb{w})$. *Compute* $[A]_1 = [\alpha]_1 + r[\delta]_1 + \sum_{i\in[m]} z_i[\mathbf{u}_i]_1$
  4. *Compute* $[B]_2 = [\beta]_2 + s[\delta]_2 + \sum_{i\in[m]} z_i[\mathbf{v}_i]_2$
  5. *Compute* $[C]_1 = s[A]_1 + r[B]_1 - rs[\delta]_1 + \sum_{i\in[k+1,m]} z_i[\mathbf{t}^{\mathsf{p}}_{i-k}]_1 + [Q]_1$
  6. *Output* $\pi = ([A]_1, [C]_1, [B]_2)$
- ***Groth16***$.\mathcal{V}(\mathbf{vk}, \mathbb{x}, \pi) \to 0/1$
  1. *Parse* π *as* $([A]_1, [C]_1, [B]_2) \in \mathbb{G}_1^2 \times \mathbb{G}_2$
  2. *Output 1 if* $[A]_1 \cdot [B]_2 = [\alpha]_1 \cdot [\beta]_2 + \left(\sum_{i=0}^{k-1} z_i[\mathbf{t}^{\mathsf{v}}_i]_1\right) \cdot [\gamma]_2 + [C]_1 \cdot [\delta]_2$. *Otherwise, output 0.*

for all $i \in [m]$. In $Groth16.\mathcal{P}$, the existence of $z = (1, \mathbb{x}, \mathbb{w}) \in \mathbb{F}^m$ such that $(A \cdot z) \circ (B \cdot z) = C \cdot z$ [2] is equivalent to the existence of a degree $n-2$ quotient polynomial $q(X)$ such that the following equality holds,

$$A(X) \cdot B(X) - C(X) = q(X) \cdot v_{\mathbb{H}}(X) \tag{1}$$

where

$$\begin{aligned} A(X) &:= \sum\nolimits_{i\in[m]} z_i u_i(X) = \sum\nolimits_{i\in[n]} a_i \ell_i(X), \\ B(X) &:= \sum\nolimits_{i\in[m]} z_i v_i(X) = \sum\nolimits_{i\in[n]} b_i \ell_i(X), \\ C(X) &:= \sum\nolimits_{i\in[m]} z_i w_i(X) = \sum\nolimits_{i\in[n]} c_i \ell_i(X) \end{aligned} \tag{2}$$

and for all $i \in [n]$,

$$a_i = (\sum\nolimits_{j\in[m]} A_{i,j} z_j), \quad b_i = (\sum\nolimits_{j\in[m]} B_{i,j} z_j), \quad c_i = (\sum\nolimits_{j\in[m]} C_{i,j} z_j) \tag{3}$$

Note that compared to the original version in [Gro16], $([\mathbf{u}]_1, [\mathbf{v}]_1, [\mathbf{v}]_2)$ are added to $\mathbf{pk}$ to improve the efficiency of the prover. It does not affect the security

[2] We slightly abuse the notation by using A, B, C for the R1CS matrices, and $[A]_1, [C]_1 \in \mathbb{G}_1, [B]_2 \in \mathbb{G}_2$ for the proof elements derived from them.

of the protocol, as they can be computed from the original $\mathbf{pk}$ without knowing the secret τ, as suggested in [Gro16].

Lemma 1. *Protocol 1 is a zk-SNARK under the q-DLOG assumption in the AGM model.*

3 Semi-Dynamic zkSNARK

3.1 Definition

Definition 1 (Semi-dynamic zk-SNARKs). *A semi-dynamic zero-knowledge succinct non-interactive argument of knowledge (semi-dynamic zk-SNARK) for an indexed relation $\mathcal{R}$ consists of the following PPT algorithms $\Psi = (\mathcal{G}, \mathcal{K}, \mathcal{P}, \mathcal{U}, \mathcal{V})$:*

- $\mathcal{G}(1^\lambda, \mathbb{i}) \to (\mathbf{pk}, \mathbf{vk})$*: On 1^λ and index $\mathbb{i}$, outputs a public key $\mathbf{pk}$ and a verifier key $\mathbf{vk}$.*
- $\mathcal{K}(\mathbf{pk}, \mathbb{x}, \mathbb{w}) \to D$*: On public key $\mathbf{pk}$, instance $\mathbb{x}$ and witness $\mathbb{w}$, outputs preprocessed data D.*
- $\mathcal{P}(\mathbf{pk}, \mathbb{x}, \mathbb{w}) \to (\pi, \mathsf{rand})$*: On public key $\mathbf{pk}$, valid extended witness $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}_\mathbb{i}$, outputs the proof π and the randomness rand.*
- $\mathcal{U}(\mathbf{pk}, D, \mathbb{x}, \mathbb{w}, \mathbb{x}', \mathbb{w}', \pi, \mathsf{rand}) \to \pi'$*: On public key $\mathbf{pk}$, valid extended witnesses $(\mathbb{x}, \mathbb{w}), (\mathbb{x}', \mathbb{w}') \in \mathcal{R}_\mathbb{i}$, proof π and randomness rand outputted by $\mathcal{P}$, preprocessed data D outputted by $\mathcal{K}$, outputs updated proof π'.*
- $\mathcal{V}(\mathbf{vk}, \mathbb{x}, \pi) \to 0/1$*: On verifier key $\mathbf{vk}$, instance $\mathbb{x}$ and proof π outputed by $\mathcal{P}$ or $\mathcal{U}$, outputs accept or reject.*

A semi-dynamic zk-SNARK Ψ satisfies the following properties:

- ***Completeness:*** *It is perfectly complete if for any $(\mathbb{i}, \mathbb{x}, \mathbb{w}), (\mathbb{i}, \mathbb{x}', \mathbb{w}') \in \mathcal{R}$:*

$$\Pr\left[\begin{array}{l|l} \mathcal{V}(\mathbf{vk}, \mathbb{x}, \pi) = 1 \wedge & (\mathbf{pk}, \mathbf{vk}) \leftarrow \mathcal{G}(1^\lambda, \mathbb{i}) \\ \mathcal{V}(\mathbf{vk}, \mathbb{x}', \pi') = 1 & D \leftarrow \mathcal{K}(\mathbf{pk}, \mathbb{x}, \mathbb{w}) \\ & (\pi, \mathsf{rand}) \leftarrow \mathcal{P}(\mathbf{pk}, \mathbb{x}, \mathbb{w}) \\ & \pi' \leftarrow \mathcal{U}(\mathbf{pk}, D, \mathbb{x}, \mathbb{w}, \mathbb{x}', \mathbb{w}', \pi, \mathsf{rand}) \end{array}\right] = 1$$

- ***Knowledge Soundness:*** *It is knowledge sound if for any PPT adversary $\mathcal{A}$ and for any $\mathbb{i}$ there exists a PPT extractor $\mathcal{E}_\mathcal{A}$ such that the following probability is $\leq \mathsf{negl}(\lambda)$*

$$\Pr\left[\mathcal{V}(\mathbf{vk}, \mathbb{x}, \pi) = 1 \wedge (\mathbb{i}, \mathbb{x}, \mathbb{w}) \notin \mathcal{R} \;\middle|\; \begin{array}{l} (\mathbf{pk}, \mathbf{vk}) \leftarrow \mathcal{G}(1^\lambda, \mathbb{i}) \\ ((\mathbb{x}, \pi); \mathbb{w}) \leftarrow (\mathcal{A} \parallel \mathcal{E}_\mathcal{A})(\mathbf{pk}, \mathbf{vk}) \end{array}\right]$$

- ***Perfect Zero-Knowledge:*** *It is zero-knowledge if there exists a PPT simulator $\mathcal{S}$ such that for any distinguisher $\mathcal{D}$, for any $(\mathbb{i}, \mathbb{x}, \mathbb{w}), (\mathbb{i}, \mathbb{x}', \mathbb{w}') \in \mathcal{R}$:*

$$\begin{aligned} &\Pr\left[\mathcal{D}(\mathbf{vk}, \mathbb{x}, \tilde{\pi}) = 1 \;\middle|\; \begin{array}{l} (\mathbf{pk}, \mathbf{vk}) \leftarrow \mathcal{S}(1^\lambda, \mathbb{i}) \\ \tilde{\pi} \leftarrow \mathcal{S}(\mathbf{pk}, \mathbf{vk}, \mathbb{x}) \end{array}\right] \\ =&\Pr\left[\mathcal{D}(\mathbf{vk}, \mathbb{x}, \pi) = 1 \;\middle|\; \begin{array}{l} (\mathbf{pk}, \mathbf{vk}) \leftarrow \mathcal{G}(1^\lambda, \mathbb{i}) \\ \pi \leftarrow \mathcal{P}(\mathbf{pk}, \mathbb{x}, \mathbb{w}) \end{array}\right] \end{aligned}$$

and

$$\Pr\left[\mathcal{D}(\mathbf{vk},\mathbb{x},\pi')=1 \;\middle|\; \begin{array}{l}(\mathbf{pk},\mathbf{vk})\leftarrow\mathcal{G}(1^\lambda,\mathbb{i})\\ \pi'\leftarrow\mathcal{P}(\mathbf{pk},\mathbb{x}',\mathbb{w}')\end{array}\right]$$

$$=\Pr\left[\mathcal{D}(\mathbf{vk},\mathbb{x},\pi')=1 \;\middle|\; \begin{array}{l}(\mathbf{pk},\mathbf{vk})\leftarrow\mathcal{G}(1^\lambda,\mathbb{i})\\ (\pi,\mathsf{rand})\leftarrow\mathcal{P}(\mathbf{pk},\mathbb{x},\mathbb{w})\\ D\leftarrow\mathcal{K}(\mathbf{pk},\mathbb{x},\mathbb{w})\\ \pi'\leftarrow\mathcal{U}(\mathbf{pk},D,\mathbb{x},\mathbb{w},\mathbb{x}',\mathbb{w}',\pi,\mathsf{rand})\end{array}\right]$$

- ***Succinctness:*** *The proof size $|\pi|$ is sublinear in the witness size $|\mathbb{w}|$.*
- ***Updatability:*** *Let $|\Delta_\mathbb{x}|, |\Delta_\mathbb{w}|$ denote the Hamming distances between $\mathbb{x}, \mathbb{x}'$ and $\mathbb{w}, \mathbb{w}'$. If $|\Delta_\mathbb{x}| + |\Delta_\mathbb{w}| = o(|\mathbb{x}| + |\mathbb{w}|)$, the running time of update $\mathcal{U}$ should be sublinear in the running time of the prover $\mathcal{P}$.*

Definition 1 above is adapted from the definition of dynamic zkSNARKs in [WPSP25b]. We combine both the prover key and the updater key into the public key $\mathbf{pk}$ as they are both public and are almost the same. In our definition, we have a preprocessing algorithm $\mathcal{K}$ that takes the original extended witness $(\mathbb{x}, \mathbb{w})$ as input and computes the preprocessed data D. New proofs are always computed using this preprocessed data of the original witness in the semi-dynamic setting. Note that we ask the prover to output the randomness rand used to generate the proof. This is necessary to re-randomize the proof in sublinear time in our update algorithm. It is not sent to the verifier, and can be omitted if zero-knowledge is not needed. The update algorithm $\mathcal{U}$ takes both the original witness and the new witness as the input, and we assume that they are valid witnesses in the indexed relation. In practice, the new witness can often be computed in sublinear time by traversing the related gates of the updated input in a circuit. Moreover, the size of $\mathbf{uk}, D$ and the witnesses are linear, but the running time of $\mathcal{U}$ can be sublinear with a linear size storage. This is the same model as the sublinear prover time of lookup arguments such as [EFG22], and the logarithmic prover time of the classical Merkle tree [Mer87]. The notion of knowledge soundness extends naturally to the updater, where the adversary is allowed to invoke the update algorithm. Finally, our definition of zero-knowledge requires that the proof generated by $\mathcal{U}$ is indistinguishable from the proof generated by $\mathcal{P}$, which is stronger than the definition in [WPSP25b].

3.2 Semi-Dynamic zkSNARK based on Groth16

In subsequent sections, we present our construction of semi-dynamic zkSNARK on top of Groth16.

Distance between witnesses. By the definition of R1CS, the difference between the original witness $z = (\mathbb{x}, \mathbb{w})$ and a new witness $z' = (\mathbb{x}', \mathbb{w}')$ is defined as $\Delta^z = (z'_i - z_i)_{i\in[m]}$. Δ^z can be represented by its dense form $(i, \Delta^z_i)_{i\in I_z}$, where I_z is the set of indices for which $\Delta^z_i \neq 0$. The Hamming distance between z and z' is denoted as $|\Delta_z| = |I_z| = d$.

In our protocol, we also need the differences

$$\Delta^A = A\cdot(z'-z),\ \Delta^B = B\cdot(z'-z),\ \Delta^C = C\cdot(z'-z),$$

with their corresponding non-zero indices as I_A, I_B, I_C. Their size may not be sublinear even if $|\Delta_z|$ is. This could happen when the i-th column of A, B or C is dense for some $i \in I_z$. In practice, when the computation is modeled as a circuit, this scenario corresponds to the case where one wire connects to many gates in the circuit. For the simplicity of stating the asymptotic complexity, we assume that $|\Delta_A|, |\Delta_B|, |\Delta_C| = O(d)$. However, our protocols still work without this assumption, and in Table 2 of Section 5, we provide the precise calculation of the number of group operations in our schemes. The same assumption is implicit in [WPSP25b]. In fact, their scheme uses the Plonkish arithmetization, which has more redundancy than R1CS and a change in one wire value may lead to more changes in the extended witness.

Updating the proof. After updating the extended witness, our goal is to update the Groth16 proof $\pi = ([A]_1, [B]_2, [C]_1)$ to $\pi' = ([A']_1, [B']_2, [C']_1)$ for z'. Recall that $[A]_1$ and $[B]_2$ in Step 3 and Step 4 of Protocol 1 are computed as:

$$[A]_1 = [\alpha]_1 + r[\delta]_1 + \sum\nolimits_{i\in[m]} z_i[\mathbf{u}_i]_1, \quad [B]_2 = [\beta]_2 + s[\delta]_2 + \sum\nolimits_{i\in[m]} z_i[\mathbf{v}_i]_2,$$

which are linear combinations of the public key **pk** with the extended witness and the randomness. They can be easily updated with the new witness. To achieve zero-knowledge, the randomness must be refreshed, thus the new proof becomes:

$$[A']_1 = [\alpha]_1 + r'[\delta]_1 + \sum\nolimits_{i\in[m]} z'_i[\mathbf{u}_i]_1, \quad [B']_2 = [\beta]_2 + s'[\delta]_2 + \sum\nolimits_{i\in[m]} z'_i[\mathbf{v}_i]_2$$

which can be computed as:

$$[A']_1 = [A]_1 + (r'-r)[\delta]_1 + \sum\nolimits_{i\in I_z} \Delta^z_i[\mathbf{u}_i]_1, [B']_2 = [B]_2 + (s'-s)[\delta]_2 + \sum\nolimits_{i\in I_z} \Delta^z_i[\mathbf{v}_i]_2$$

The update consists of $O(|\Delta_z|) = O(d)$ group operations.

Similarly, recall $[C]_1$ in Step 5 of Protocol 1 is computed as:

$$[C]_1 = s[A]_1 + r[B]_1 - rs[\delta]_1 + \sum\nolimits_{i\in[k,m]} z_i[\mathbf{t}^{\mathsf{p}}_{i-k}]_1 + [Q]_1$$

where $[Q]_1 = \left[\frac{q(\tau)(\tau^n-1)}{\delta}\right]_1$. Consequently, the updated proof $[C']_1$ becomes:

$$[C']_1 = s'[A']_1 + r'[B']_1 - r's'[\delta]_1 + \sum\nolimits_{i\in[k,m]} z'_i[\mathbf{t}^{\mathsf{p}}_{i-k}]_1 + [Q']_1$$

and the difference from $[C]_1$ is given by $[\Delta^C_{\text{lin}}]_1 + ([Q']_1 - [Q]_1)$ where

$$[\Delta^C_{\text{lin}}]_1 = s'[A']_1 - s[A]_1 + r'[B']_1 - r[B]_1 - (r's' - rs)[\delta]_1 + \sum\nolimits_{i\in[k,m]\cap I_z} \Delta^z_i[\mathbf{t}^{\mathsf{p}}_{i-k}]_1$$

All terms within $[\Delta^C_{\text{lin}}]_1$ can be computed efficiently. The term $s'[A']_1$ can be directly computed using $[A']_1$ from previous steps. Using the same approach to compute and update the term $[B]_2$, the prover also computes and updates $[B']_1$ which is used as a component in the update of $[C]_1$. The complexity of computing

all these linear terms is $O(d)$ group operations. Note that this is the step where the updater $\mathcal{U}$ does need to know both the old randomness r, s and the new randomness r', s' in the protocol. That is why $\mathcal{P}$ needs to pass the randomness to $\mathcal{U}$ in our definition.

Quotient term. The main challenge to update the proof lies in updating the quotient term:

$$[Q']_1 - [Q]_1 = \left[\frac{q(\tau)(\tau^n - 1)}{\delta}\right]_1 - \left[\frac{q'(\tau)(\tau^n - 1)}{\delta}\right]_1 = \left[\frac{(q'(\tau) - q(\tau))(\tau^n - 1)}{\delta}\right]_1.$$

Recall that $q(X)$ is defined as:

$$A(X) \cdot B(X) - C(X) = q(X) \cdot v_{\mathbb{H}}(X)$$

where the definition of $A(X)$, $B(X)$ and $C(X)$ can be found in Equation (2) and Equation (3). To use the Lagrange basis, we define the update polynomials as:

$$\Delta^A(X) = A'(X) - A(X) = \sum\nolimits_{i \in I_A} \Delta_i^A \ell_i(X)$$

[Ty:] and $\Delta^B(X), \Delta^C(X)$ similarly, where $\Delta^A = A \cdot \Delta^z, \Delta^B = B \cdot \Delta^z, \Delta^C = C \cdot \Delta^z \in \mathbb{F}^n$ for all $i \in [n]$. Therefore, when updating the quotient polynomial,

$$\begin{aligned}
&(A'(x) \cdot B'(X) - C'(X)) - (A(x) \cdot B(X) - C(X)) \\
&= (A(X) + \Delta^A(X)) \cdot (B(X) + \Delta^B(X)) - A(X) \cdot B(X) - \Delta^C(X) \\
&= \Delta^A(X) \cdot B(X) + \Delta^B(X) \cdot A(X) + \Delta^A(X) \cdot \Delta^B(X) - \Delta^C(X)
\end{aligned}$$

Because the degree of $\Delta^C(X)$ is $n-1$, it does not affect the quotient polynomial and its role is making the remainder polynomial zero (i.e., the witness remains valid after the update). However, the first three terms are all nonlinear in the witness, making them difficult to compute efficiently.

We begin by handling the cross term $\Delta^A(X) \cdot \Delta^B(X)$ in Section 3.3. We then address the two projection terms $\Delta^A(X) \cdot B(X)$ and $\Delta^B(X) \cdot A(X)$ in Section 3.4, and present the full protocol in Section 3.5.

3.3 Cross Term

The cross-term can be expressed as:

$$\begin{aligned}
\Delta^A(X) \cdot \Delta^B(X) &= \left(\sum\nolimits_{i \in I_A} \Delta_i^A \ell_i(X)\right) \cdot \left(\sum\nolimits_{i \in I_B} \Delta_i^B \ell_i(X)\right) \\
&= \sum\nolimits_{(i,j) \in (I_A, I_B)} \Delta_i^A \Delta_j^B \ell_i(X) \ell_j(X)
\end{aligned}$$

Although the summation only involves $O(d^2)$ term, computing the cross term remains challenging. For each $(i, j) \in (I_A, I_B)$, we cannot efficiently derive

$\frac{\ell_i(X)\ell_j(X)}{v_{\mathbb{H}}(X)}$ from $\{X^i\}_{i\in[n]}$. It is possible to precompute all $\{\frac{\ell_i(X)\ell_j(X)}{v_{\mathbb{H}}(X)}\}_{(i,j)\in[n]^2}$ in the preprocessing phase, but it would take $O(n^2)$ time and storage, which is impractical for large n.

Our key insight is that the family of products $\ell_i(X)\ell_j(X)$ for every $(i,j) \in [n]^2$ exhibits a highly structured quotient form. We formalize this observation in the following theorem:

Theorem 1 (Quotient Decomposition).

$$\forall i \neq j \in [n],\ \ell_i(X)\cdot\ell_j(X) = \phi_{i,j}(X)\cdot(X^n-1)\,,$$
$$\forall i \in [n],\ \ell_i(X)\cdot\ell_i(X) = \phi_{i,i}(X)\cdot(X^n-1) + \ell_i(X)$$

where

$$\phi_{i,j}(X) = \mathbf{w}_{j-i}\cdot\ell_i(X) + \mathbf{w}_{i-j}\cdot\ell_j(X)\,, \tag{4}$$
$$\phi_{i,i}(X) = -\textstyle\sum_{j\in[n]/i}\phi_{i,j}(X) \tag{5}$$

$$\mathbf{w} = \left(\tfrac{\omega^i}{n(1-\omega^i)}\right)_{i=0}^{n-1},\quad \mathbf{w}' = \left(\tfrac{\omega^{-i}}{n(1-\omega^{-i})}\right)_{i=0}^{n-1} \in \mathbb{F}^n \tag{6}$$

The proof of the the theorem is provided in the full version. This theorem allows decomposing the quotient of any product of polynomials into linear combinations of $\{\phi_{i,j}(X)\}_{(i,j)\in[n]^2}$, which can be further decomposed into linear combinations of $\{\ell_i(X)\}_{i\in[n]}$ and $\{\phi_{i,i}(X)\}_{i\in[n]}$. Using Theorem 1, we can decompose the quotient of the cross term as:

$$\begin{aligned}\Delta^A(X)\cdot\Delta^B(X) &= \textstyle\sum_{(i,j)\in(I_A,I_B)}\Delta_i^A\Delta_j^B\ell_i(X)\ell_j(X)\\ &= \left(\textstyle\sum_{(i,j)\in(I_A,I_B)}\Delta_i^A\Delta_j^B\phi_{i,j}(X)\right)v_{\mathbb{H}}(X) + \textstyle\sum_{i\in I_A\cap I_B}\Delta_i^A\Delta_i^B\ell_i(X)\end{aligned}$$

Therefore, the quotient of the cross term can be expressed as:

$$\begin{aligned}q^{\text{cross}}(X) &= \textstyle\sum_{(i,j)\in(I_A,I_B)}\Delta_i^A\Delta_j^B\phi_{i,j}(X)\\ &= \textstyle\sum_{i\in I}\Delta_i^A\Delta_i^B\phi_{i,i}(X) + \sum_{i\in I}\sum_{j\in I/i}(\Delta_i^A\Delta_j^B + \Delta_i^B\Delta_j^A)\cdot\mathbf{w}_{j-i}\cdot\ell_i(X).\end{aligned} \tag{7}$$

Computing this term only takes $O(|I|)$ group operations and $O(|I|^2)$ field operations where $I = I_A \cup I_B$ with preprocessing of $\{\ell_i(X)\}_{i\in[n]}$ and $\{\phi_{i,i}(X)\}_{i\in[n]}$.

Reducing the number of field operations via sparse convolution. We can further reduce the complexity of field operations to $O(|I|\log^2|I|)$ by refining the decomposition to write the quotient as a linear combination of $\ell_i(X)$ for $i \in I$:

$$\begin{aligned}q^{\text{cross}}(X) = &\textstyle\sum_{i\in I}\Delta_i^A\Delta_i^B\phi_{i,i}(X) +\\ &\textstyle\sum_{i\in I}\left(\Delta_i^A\sum_{j\in I/i}(\Delta_j^B\cdot\mathbf{w}'_{i-j}) + \Delta_i^B\sum_{j\in I/i}(\Delta_j^A\cdot\mathbf{w}'_{i-j})\right)\cdot\ell_i(X)\end{aligned} \tag{8}$$

In the equation above, the quadratic field operation cost comes from computing $\sum_{j\in I/i} \Delta^B_j \cdot \mathbf{w}'_{i-j}$ and $\sum_{j\in I/i} \Delta^A_j \cdot \mathbf{w}'_{i-j}$ for all $i \in I$, which takes $O(|I|^2)$ time. These sums can be interpreted as convolutions of the sparse vectors Δ^A and Δ^B with a special fixed vector $\mathbf{w}'$ defined in Equation (6), where the input vectors are nonzero only on indices in I and only the output entries corresponding to I are needed. We denote these operations as $\mathrm{SparseConv}_{\mathbf{w}'}(\mathbf{a}, I)$ and $\mathrm{SparseConv}_{\mathbf{w}'}(\mathbf{b}, I)$.

We present an algorithm to compute such sparse convolution in $O(|I|\log^2|I|)$ field operations in the full version.The algorithm is a special and simplified variant of the algorithms in [GGS87] for the sparse convolution.

Keeping the same prover key as Groth16. The protocol above assumes the existence of $\left[\frac{\ell_i(\tau)(\tau^n-1)}{\delta}\right]_1$ for $i \in [n]$ in $\mathbf{pk}$. However, in the original Protocol 1 of Groth16, the trusted setup outputs $[\boldsymbol{\tau}]_1 = \left(\left[\frac{\tau^i(\tau^n-1)}{\delta}\right]_1\right)_{i=0}^{n-2}$, while the highest degree of $\ell_i(\tau)$ is $n-1$. Therefore, we cannot compute $\left[\frac{\ell_i(\tau)(\tau^n-1)}{\delta}\right]_1$ from $[\boldsymbol{\tau}]_1$ without knowing the secret τ. It is not hard to re-prove the security of Groth16 by providing an extra element $\left[\frac{\tau^{n-1}(\tau^n-1)}{\delta}\right]_1$ in $\mathbf{pk}$.

However, we argue that this is unnecessary, since the degree of the quotient polynomial $q(X)$ remains $n-2$, and the highest degree of $\ell_i(X)$ must cancel out with each other. More specificly, all quotients can be decomposed into a linear combination of $\{\phi_{i,j}(X)\}_{i,j\in[n]}$. For all $i \neq j \in [n]$, the highest degree term of $\phi_{i,j}(X)$ is zero since by Equation (4)

$$\frac{\omega^{j-i}}{n(1-\omega^{j-i})}\frac{\omega^i}{n}X^{n-1} + \frac{\omega^{i-j}}{n(1-\omega^{i-j})}\frac{\omega^j}{n}X^{n-1} = 0$$

Moreover, for every $i \in [n]$, we have that $\phi_{i,i}(X) = -\sum_{j\in[n]/i}\phi_{i,j}(X)$ as shown in Equation (5), so that the highest degree term of $\phi_{i,i}(X)$ is also zero. Therefore, it is enough to only use *truncated lagrange basis* which are defined as:

$$\bar{\ell}_i(X) = \frac{1}{n}\cdot\sum\nolimits_{j\in[n-1]}(\omega^i)^{-j}\cdot X^j$$

Efficient preprocessing. It remains to show how to efficiently compute

$$[\bar{\boldsymbol{\ell}}]_1 := \left(\left[\frac{\bar{\ell}_i(\tau)(\tau^n-1)}{\delta}\right]\right)_{i=0}^{n-1}, \quad [\mathbf{p}]_1 := \left(\left[\frac{\phi_{i,i}(\tau)(\tau^n-1)}{\delta}\right]\right)_{i=0}^{n-1}$$

from

$$[\bar{\boldsymbol{\tau}}]_1 = ([\boldsymbol{\tau}]_1, 0_{\mathbb{G}_1}) \in \mathbb{G}^n, \quad [\boldsymbol{\tau}]_1 = \left(\left[\frac{\tau^i(\tau^n-1)}{\delta}\right]_1\right)_{i=0}^{n-2} \in \mathbf{pk}$$

– Preprocessing Truncated Lagrange Basis. The Lagrange basis can be derived from the monomial basis using the inverse DFT as follows

$$(\ell_i(X))_{i=0}^{n-1} = \mathrm{iDFT}((X^i)_{i=0}^{n-1})$$

By linearity of iDFT, we can remove the degree $n-1$ term in monomials to compute the truncated Lagrange basis, and multiply them by the constant

$\frac{\tau^n-1}{\delta}$. Therefore, we can compute $[\bar{\boldsymbol{\ell}}]_1 = \mathrm{iDFT}([\bar{\boldsymbol{\tau}}]_1)$ in $O(n \log n)$ time. The proofs are deferred to the full version.

– Preprocessing Diagonals. The naive approach to compute $\phi_{i,i}(X)$ for all $i \in [n]$ would take $O(n^2)$ time by taking an n-size summation for each term. By exploiting their convolution structure as follows

$$\phi_{i,i}(X) = -(\textstyle\sum_{i\in[n]} \mathbf{w}_i) \cdot \ell_i(X) - \sum_{j\in[n]/i} \mathbf{w}_{i-j} \cdot \ell_j(X),$$

we can compute $[\mathbf{p}]_1 = -(\sum_{i\in[n]} \mathbf{w}_i) \cdot [\bar{\boldsymbol{\ell}}]_1 - \mathbf{w} * [\bar{\boldsymbol{\ell}}]_1$ in $O(n \log n)$ time. The proofs are deferred to the full version.

Complexity analysis. The contribution of the cross term to the updated proof $[C']_1$ is given by

$$[\Delta^C_{\text{cross}}]_1 = \sum\nolimits_{i\in I_A \cup I_B} (\Delta^A_i \tilde{\Delta}^B_i + \Delta^B_i \tilde{\Delta}^A_i)[\bar{\boldsymbol{\ell}}_i]_1 + \Delta^A_i \Delta^B_i [\mathbf{p}_i]_1$$

where $\tilde{\Delta}^A = \mathrm{SparseConv}_{\mathbf{w}'}(\Delta^A, I_A \cup I_B)$ and $\tilde{\Delta}^B = \mathrm{SparseConv}_{\mathbf{w}'}(\Delta^B, I_A \cup I_B)$. The complexity is $O(d)\mathbb{G} + O(d \log^2 d)\mathbb{F}$ with $O(n \log n)\mathbb{G}$ preprocessing time.

3.4 Projection Terms

The projection terms can be written as:

$$\begin{aligned}
&\Delta_A(X) \cdot B(X) + \Delta_B(X) \cdot A(X) \\
&= \Big(\sum\nolimits_{i\in I_A} \Delta^A_i \ell_i(X)\Big) \cdot B(X) + \Big(\sum\nolimits_{i\in I_B} \Delta^B_i \ell_i(X)\Big) \cdot A(X) \\
&= \sum\nolimits_{i\in I_A} \Delta^A_i \cdot (\ell_i(X) \cdot B(X)) + \sum\nolimits_{i\in I_B} \Delta^B_i \cdot (\ell_i(X) \cdot A(X))
\end{aligned} \tag{9}$$

The formula is very similar to the sublinear lookup argument in [EFG22]. With a one-time preprocessing on $A(X), B(X)$, the quotient of the projection terms can be computed efficiently. We further optimize this preprocessing by expressing all projection quotients in convolution form. This is summarized in Theorem 2, building on Theorem 1.

Theorem 2 (Projection Quotient). *For each $i \in [n]$, the i-th projection quotient $q^a_i(X)$ for polynomial the $A(X) = \sum_{i\in[n]} a_i \ell_i(X)$ is defined by*

$$A(X)\ell_i(X) = q^a_i(X) v_{\mathbb{H}}(X) + a_i \ell_i(X)$$

The vector of all projection quotients $(q^a_i(X))^{n-1}_{i=0}$ can be expressed as

$$(q^a_i(X))^{n-1}_{i=0} = \mathbf{w} * (\mathbf{a} \circ (\ell_i(X))^{n-1}_{i=0}) + (\mathbf{w}' * \mathbf{a}) \circ (\ell_i(X))^{n-1}_{i=0} + \mathbf{a} \circ (\phi_{i,i}(X))^{n-1}_{i=0}$$

where $\mathbf{w}$ and $\mathbf{w}'$ are defined in Theorem 1.

We can further simplify Equation (9) using $\{q_i^a(X)\}_{i\in[n]}$ defined in Theorem 2 into the following

$$\Delta_A(X)\cdot B(X)+\Delta_B(X)\cdot A(X)=\sum\nolimits_{i\in I_A}\Delta_i^A\cdot q_i^B(X)+\sum\nolimits_{i\in I_B}\Delta_i^B\cdot q_i^A(X)$$

With preprocessing of projection quotients, the projection term can be computed in $O(n)$ group operations.

Keeping the same prover key as Groth16. Similar to the cross term, the prover key can be made the same as Groth16 as the monomials with degree $n-1$ in the Lagrange polynomials always cancel out with each other.

Efficient preprocessing. The projection quotients $[\mathbf{q}^{\mathsf{a}}]_1 := \left(\left[\frac{q_i^a(\tau)(\tau^n-1)}{\delta}\right]_1\right)_{i=0}^{n-1}$ and $[\mathbf{q}^{\mathsf{b}}]_1 := \left(\left[\frac{q_i^b(\tau)(\tau^n-1)}{\delta}\right]_1\right)_{i=0}^{n-1}$ can be computed efficiently in $O(n\log n)$ time by applying Theorem 2 as follows

$$[\mathbf{q}^{\mathsf{a}}]_1=\mathbf{w}*(\mathbf{a}\circ[\bar{\boldsymbol{\ell}}]_1)+(\mathbf{w}'*\mathbf{a})\circ[\bar{\boldsymbol{\ell}}]_1+\mathbf{a}\circ[\mathbf{p}]_1$$

$$[\mathbf{q}^{\mathsf{b}}]_1=\mathbf{w}*(\mathbf{b}\circ[\bar{\boldsymbol{\ell}}]_1)+(\mathbf{w}'*\mathbf{b})\circ[\bar{\boldsymbol{\ell}}]_1+\mathbf{b}\circ[\mathbf{p}]_1$$

Our preprocessing of projection quotients is around 2× faster than [FK23] as we use circular convolutions instead of linear convolutions. We highlight that our preprocessing algorithm is not only concretely faster but also conceptually simpler than the one in [FK23], and the core insight to derive this simple preprocessing algorithm is still the perspective of quotient decomposition.

Complexity analysis. The contribution of the projection terms to the updated proof $[C']_1$ is

$$[\Delta_{\text{proj}}^C]_1=\sum\nolimits_{i\in I_A}\Delta_i^A[\mathbf{q}_i^{\mathsf{b}}]_1+\sum\nolimits_{i\in I_B}\Delta_i^B[\mathbf{q}_i^{\mathsf{a}}]_1$$

The complexity is $O(d)\mathbb{G}$ with $O(n\log n)\mathbb{G}$ preprocessing time.

3.5 Putting Everything Together

The full protocol of our semi-dynamic zkSNARK is presented in Protocol 2. In $\mathit{SemiDynark}.\mathcal{G}$, we have additional elements $[\bar{\boldsymbol{\ell}}]_1$ and $[\mathbf{p}]_1$ in the public parameters of our protocol over Groth16. All of them can be computed using the public key of Groth16 without knowing the secret τ. Therefore, our protocol can be used on top of Groth16 without rerunning the trusted setup. Alternatively, these elements can be computed more efficiently with τ in a trusted setup, and they do not depend on the R1CS or the witness. We summarize this result in the following theorem with the proof deferred to the full version.

Theorem 3. *Protocol 2 is a semi-dynamic zkSNARK by Definition 1 under the q-DLOG assumption in the AGM.*

Protocol 2 (Semi-Dynamic zk-SNARK)

- *SemiDynark.*$\mathcal{G}(1^\lambda, \mathtt{i}) \to (\mathbf{pk}, \mathbf{vk})$
  1. *Groth*16.$\mathbf{pk} \leftarrow$ *Groth*16.$\mathcal{G}(1^\lambda, \mathtt{i})$.
  2. *Let* $\mathbf{w} = \left(\frac{\omega^i}{1-\omega^i}\right)_{i=0}^{n-1} \in \mathbb{F}^n$ *and* $\mathbf{w}' = \left(\frac{\omega^{-i}}{1-\omega^{-i}}\right)_{i=0}^{n-1} \in \mathbb{F}^n$.
  3. *Compute* $[\bar{\boldsymbol{\ell}}]_1 = iDFT([\bar{\boldsymbol{\tau}}]_1)$
     where $[\bar{\boldsymbol{\tau}}]_1 = ([\boldsymbol{\tau}]_1, 0_{\mathbb{G}_1})$ *and* $[\boldsymbol{\tau}]_1 \in$ *Groth*16.$\mathbf{pk}$
  4. *Compute* $[\mathbf{p}]_1 = -\left(\left(\sum_{i\in[n]} \mathbf{w}_i\right) \cdot [\bar{\boldsymbol{\ell}}]_1 + \mathbf{w} * [\bar{\boldsymbol{\ell}}]_1\right)$
  5. *Output* $\mathbf{pk} = (Groth16.\mathbf{pk}, [\bar{\boldsymbol{\ell}}]_1, [\mathbf{p}]_1), \mathbf{vk} = Groth16.\mathbf{vk}$
- *SemiDynark.*$\mathcal{K}(\mathbf{pk}, \mathbb{x}, \mathbb{w}) \to D$*:*
  1. *Compute* $\mathbf{a} = A \cdot z, \mathbf{b} = B \cdot z, \mathbf{c} = C \cdot z \in \mathbb{F}^n$ *where* $z = (\mathbb{x}, \mathbb{w})$
  2. *Compute* $[\mathbf{q}^{\mathsf{a}}]_1 = \mathbf{w} * (\mathbf{a} \circ [\bar{\boldsymbol{\ell}}]_1) + (\mathbf{w}' * \mathbf{a}) \circ [\bar{\boldsymbol{\ell}}]_1 + \mathbf{a} \circ [\mathbf{p}]_1$
  3. *Compute* $[\mathbf{q}^{\mathsf{b}}]_1 = \mathbf{w} * (\mathbf{b} \circ [\bar{\boldsymbol{\ell}}]_1) + (\mathbf{w}' * \mathbf{b}) \circ [\bar{\boldsymbol{\ell}}]_1 + \mathbf{b} \circ [\mathbf{p}]_1$
  4. *Output* $D = ([\mathbf{q}^{\mathsf{a}}]_1, [\mathbf{q}^{\mathsf{b}}]_1)$
- *SemiDynark.*$\mathcal{P}(\mathbf{pk}, \mathbb{x}, \mathbb{w}) \to (\pi, \mathsf{rand})$*:*
  1. $\pi \leftarrow Groth16.\mathcal{P}(Groth16.\mathbf{pk}, \mathbb{x}, \mathbb{w})$ *where* $\mathsf{rand} = (r, s)$ *used in* $Groth16.\mathcal{P}$.
  2. *Output* (π, rand).
- *SemiDynark.*$\mathcal{U}(\mathbf{pk}, D, \mathbb{x}, \mathbb{w}, \mathbb{x}', \mathbb{w}', \pi, \mathsf{rand}) \to \pi'$*:*
  1. *Let* $z = (\mathbb{x}, \mathbb{w}), z' = (\mathbb{x}', \mathbb{w}')$ *and* $\Delta^z = z' - z$.
  2. *Let* $\Delta^A = A \cdot \Delta^z, \Delta^B = B \cdot \Delta^z \in \mathbb{F}^n$
  3. *Compute* $[A']_1 = [A]_1 + (r' - r)[\delta]_1 + \sum_{i\in I_z} \Delta_i^z [\mathbf{u}_i]_1$.
  4. *Compute* $[B']_2 = [B]_2 + (s' - s)[\delta]_2 + \sum_{i\in I_z} \Delta_i^z [\mathbf{v}_i]_2$.
  5. *Compute* $[\Delta^C_{lin}]_1 = (s'[A']_1 + s[A]_1) + (r'[B']_1 - r[B]_1) - (r's' - rs)[\delta]_1 + \sum_{i\in[k,m]\cap I_z} \Delta_i^z [\mathbf{t}_i^{\mathsf{p}}]_1$
  6. *Compute* $[\Delta^C_{cross}]_1 = \sum_{i\in I_A \cup I_B} (\Delta_i^A \tilde{\Delta}_i^B + \Delta_i^B \tilde{\Delta}_i^A)[\bar{\boldsymbol{\ell}}_i]_1 + \Delta_i^A \Delta_i^B [\mathbf{p}_i]_1$
     where $\tilde{\Delta}^A = SparseConv_{\mathbf{w}'}(\Delta^A, I_A \cup I_B)$ *and* $\tilde{\Delta}^B = SparseConv_{\mathbf{w}'}(\Delta^B, I_A \cup I_B)$
  7. *Compute* $[\Delta^C_{proj}]_1 = \sum_{i\in I_A} \Delta_i^A [\mathbf{q}_i^{\mathsf{b}}]_1 + \sum_{i\in I_B} \Delta_i^B [\mathbf{q}_i^{\mathsf{a}}]_1$
  8. *Compute* $[C']_1 = [C]_1 + [\Delta^C_{lin}]_1 + [\Delta^C_{cross}]_1 + [\Delta^C_{proj}]_1$
  9. *Output* $\pi' = ([A']_1, [C']_1, [B']_2)$
- *SemiDynark.*$\mathcal{V}(\mathbf{vk}, \mathbb{x}, \pi) \to 0/1$*:*
  1. *Output* $Groth16.\mathcal{V}(\mathbf{vk}, \mathbb{x}, \pi)$

Complexity. Combining the complexity analyses above, the update time of *SemiDynark* is $O(d)\mathbb{G} + O(d \log^2 d)\mathbb{F}$, after a preprocessing on the original witness of complexity $O(n \log n)\mathbb{G}$. The prover time is $O(n \log n)\mathbb{F} + O(n)\mathbb{G}$, the proof size is $O(1)$ and the verifier time is $O(1)$[3].

Improving the prover time of Groth16. With our better understanding of quotient polynomials, we also propose a better algorithm to generate the Groth16 proof with only 4 FFTs and we leave it to the full version.

[3] Assuming that the size of the public input $|\mathbb{x}|$ is constant.

4 Fully-Dynamic Groth16

Semi-dynamic zkSNARKs allow efficient proof updates from an initial witness $\mathbb{w}$ and its preprocessed data D. Their efficiency, however, relies on the assumption that the updated witness remains close to the initial one. They become inefficient in scenarios where a long sequence of updates makes the witness deviate significantly from the initial witness. To address the problem, in this section, we present the construction of our fully-dynamic zkSNARK.

The formal definition of fully-dynamic zkSNARKs is presented in the full version. Compared to Definition 1 of semi-dynamic zkSNARKs, the only difference is that the update algorithm $\mathcal{U}$ above also updates the preprocessed data D to D' for the updated extended witness $z' = (\mathbb{x}', \mathbb{w}')$, so that D' can be utilized for future updates based on z', making the scheme fully dynamic. Our construction also needs the updated randomness rand' to achieve zero-knowledge, and we denote it separately from D' in our definition. The completeness and the zero-knowledge incorporates a sequence of updates, but are essentially similar to Definition 1. Our definition is almost the same as the one in [WPSP25b], except that our zero-knowledge requires that the proof generated by $\mathcal{U}$ is indistinguishable from the proof generated by $\mathcal{P}$.

4.1 Fully-Dynamic zkSNARK based on Groth16

Fully-dynamic from semi-dynamic in the amortized setting. A naive approach to build fully-dynamic zkSNARKs from semi-dynamic zkSNARKs is to recompute the preprocessed data D' for the extended witness z' after its distance becomes b-far away from the initial extended witness z. This ensures that the distance of any new witness in subsequent updates is at most $O(b)$ away from the most recent preprocessed witness with the corresponding preprocessed data. Then using our semi-dynamic zkSNARK in the previous section, the update cost is at most $O(b)$ $\mathbb{G}$ and $O(b \log^2 b)$ $\mathbb{F}$. Meanwhile, the preprocessing cost is amortized to $O(\frac{n \log n}{b})$ $\mathbb{G}$ per unit-distance update. By setting $b = \sqrt{n \log n}$, the total amortized update time is $O(\frac{dn \log n}{b} + b)\mathbb{G} + O(b \log^2 b)\mathbb{F} = O(d\sqrt{n \log n})\mathbb{G} + O(\sqrt{n \log n} \log^2 n)\mathbb{F}$.

However, the amortized approach can lead to instability of the update time, which is undesirable in applications that require predictable update costs such as blockchain systems. We show how to construct fully dynamic zkSNARKs in the de-amortized setting below.

De-amortizing the updates. We adopt the ideas in [RT97] for dynamic algorithms to de-amortize the update cost. We divide a sequence of extended witnesses $(z_0, z_1, z_2, z_3, \ldots)$ into blocks $(B_0, B_1, B_2, \ldots)$, such that the distance between the first witness in block B_i, denoted by z_{B_i}, and the first witness in block B_{i+1}, denoted by $z_{B_{i+1}}$, is at most b for all blocks. If the distance of any single update is larger than b, then we simply rerun the preprocessing of our semi-dynamic zkSNARK and this case is consistent with the complexity of our fully-dynamic zkSNARK as we show below. Therefore, for simplicity, we assume

the distance of each update is less than b and the sequence can always be divided into the blocks above.

In the naive approach above for the amortized setting, the updater basically computes the full preprocessed data for the first witness z_{B_i} of every block B_i, and uses it to compute the proofs for all witnesses in the block. This computation can be de-amortized using the two algorithms below:

- Stepwise DFT. We first observe that the main computations in the preprocessing—namely the DFTs and iDFTs—are *smooth*, meaning that they can be divided into b smaller consecutive steps. We present this stepwise DFT algorithm in the full version. In step-DFT$_b(t, \mathbf{x})$, b is the number of steps to break the DFT into, $\mathbf{x}$ is the input of the DFT, and t is the index of the current step. By running step-DFT$_b(t, \mathbf{x})$ for $t = 0$ to $b - 1$, the output is exactly DFT($\mathbf{x}$). The same holds for step-iDFT.
  With this stepwise DFT algorithm, we present the preprocessing of our semi-dynamic zkSNARK in the stepwise manner in the full version d is the distance between two consecutive witnesses, and the complexity of the stepwise preprocessing is $O(\frac{dn \log n}{b})\mathbb{G}$. **state** denotes the intermediate results to keep between the steps.
  We emphasize that the basic step within the for loop of step-DFT can be fully-parallelized as long as $b \geq \log n$ which is the case in our protocol. Thus the de-amortization does not harm the parallelization of the computations as shown in Section 5.
- Block offset. To utilize the step-by-step update above, instead of computing and using the preprocessed data D_{B_i} of the first witness z_{B_i} for every block B_i, the updater uses the preprocessed data $D_{B_{i-1}}$ of the first witness $z_{B_{i-1}}$ of the previous block B_{i-1} to update the proofs for all witnesses in the current block B_i. At the same time, the updater computes the preprocessed data D_{B_i} for z_{B_i} step by step in the current block B_i. In this way, the updater *buys a block of time with a one block size distance offset*, within which it can incrementally compute the preprocessed data D_{B_i} for z_{B_i}. Once the updater completes the updates in the current block B_i, it also obtains the preprocessed data D_{B_i} of the witness z_{B_i}, which can be used to update the proofs for all witnesses in the next block B_{i+1}.
  Because the distance between $z_{B_{i-1}}$ and z_{B_i} is at most b, the distance between any witness in block B_i and $z_{B_{i-1}}$ is at most $2b$. Therefore, the update time remains $O(b)\mathbb{G} + O(b \log^2 b)\mathbb{F}$ using the preprocessed data $D_{B_{i-1}}$.

Combining the two algorithms, we obtain a fully-dynamic zkSNARK in the de-amortized setting with an update time of $O(\frac{dn \log n}{b} + b)\mathbb{G} + O(b \log^2 b)\mathbb{F}$, which is the same as the amortized cost in the naive approach. Note that the sequence of witnesses divided into blocks is merely to illustrate the idea of our construction. Our update algorithm only needs to know the new witness, the existing preprocessed data, the pre-defined parameter b and the state of the stepwise preprocessing, but not any future witnesses.

Partially updating preprocessed data. Finally, we introduce one more optimization to improve the complexity of the update time. We observe that al-

though the size of the preprocessed data for any witness is $O(n)$, the updater only uses part of it to update the proof. That is, only $O(d)$ elements in the preprocessed data D_z of witness z are used by the updater to update the proof from z to z' within a block B_i. This is exactly how we get $O(d)$ group operations in the update time of our semi-dynamic zkSNARK. This observation suggests that although we cannot compute the full preprocessed data D_z for every z, it is possible to compute the part of D_z that will be used by the updater to update the proof for z'. Performing this partial updates from $D_{B_{i-1}}$ and then using it to generate the proof of z' turns out to be more efficient than generating the proof of z' using $D_{B_{i-1}}$ directly.

To see this, recall that the D consists of projection quotients $\mathbf{q}^a$ and $\mathbf{q}^b$, where the i-th element can be written as

$$[\mathbf{q}_i^a]_1 = \sum\nolimits_{j\in[n]} \left(\mathbf{a}_j[\bar{\boldsymbol{\ell}}_j]_1 \mathbf{w}_{i-j}\right) + \left(\sum\nolimits_{j\in[n]} \mathbf{a}_j \mathbf{w}'_{i-j}\right)[\bar{\boldsymbol{\ell}}_i]_1 + \mathbf{a}_i[\mathbf{p}_i]_1\,,$$

which is a linear combination of elements in $\mathbf{a}$. It means that for each element of $[\mathbf{q}^a]_1$ used by the updater, the contribution of each change in $\mathbf{a}$ can be computed in constant time. The same applies to $[\mathbf{q}^b]_1$. We call this property the locality of the preprocessing. Based on this locality, we can efficiently compute the partial preprocessed data D_z from $D_{B_{i-1}}$. Since we only need to update $[\mathbf{q}^a]_1$ on $I_{z'-z}$ indices, the complexity is $O(db)\mathbb{G}$.

Using the partial updates, the update time becomes $O(\frac{dn\log n}{b} + db)\mathbb{G} + O(d\log^2 d)\mathbb{F}$. This is better than the complexity above as in our case that b is much larger than d. For example, when setting $b = \sqrt{n\log n}$, the complexity is reduced to $O(d\sqrt{n\log n})\mathbb{G}+O(d\log^2 d)\mathbb{F}$, from $O(d\sqrt{n\log n})\mathbb{G}+O(\sqrt{n\log n}\log^2 n)\mathbb{F}$.

Our fully-dynamic zkSNARK is presented formally in Protocol 3. Algorithms $\mathcal{G}, \mathcal{K}, \mathcal{P}, \mathcal{V}$ and $\mathbf{pk}, \mathbf{vk}$ of our fully-dynamic zkSNARK are exactly the same as our semi-dynamic zkSNARK in Protocol 2.

Theorem 4. *Protocol 3 is a fully-dynamic zkSNARK under the q-DLOG assumption in the AGM model.*

Proof sketch. The protocol is essentially an algorithm to periodically compute the preprocessed data for a new witness in the de-amortized setting. The completeness follows from the correctness of step-$\mathcal{K}$, partial-$\mathcal{K}$ and the completeness of the semi-dynamic scheme as explained above. The public key, proof and verifier algorithm remains exactly the same as *SemiDynark*. Therefore, both the knowledge soundness and the zero-knowledge follow the same argument as the proof of Theorem 3.

Compiling semi-dynamic zkSNARK to full-dynamic zkSNARK. Our protocol also suggests a way to generically compile a semi-dynamic zkSNARK to a fully-dynamic zkSNARK in the de-amortized setting. Suppose that the running time of $SemiDynark.\mathcal{K}$ is $T(\mathcal{K})$ and can be computed stepwise. Let $T(\mathcal{U})$ be the running time of $SemiDynark.\mathcal{U}$ per unit distance. By running Protocol 3 (excluding the partial update optimization in Step 2), we are able to construct

Protocol 3 (Fully-Dynamic zk-SNARK)

– $\mathcal{G}, \mathcal{K}, \mathcal{P}, \mathcal{V}$ *of FullyDynark are exactly the same as SemiDynark in Protocol 2.*
– *FullyDynark*.$\mathcal{U}(\mathbf{pk}, D, \mathbb{x}, \mathbb{w}, \mathbb{x}', \mathbb{w}', \pi, \mathsf{rand}) \to (\pi', \mathsf{rand}', D')$*:*
It maintains the following data in D*:*
- t*: index of steps*
- $z_{B_{i-1}}$*: the first witness of the previous block* B_{i-1}
- z_{B_i}*: the first witness of the current block* B_i
- *semiD: the preprocessed data for* $z_{B_{i-1}}$
- **state***: the intermediate result of preprocessing for* z_{B_i}

The updater works as follows:
1. *let* $z = (\mathbb{x}, \mathbb{w})$ *and* $z' = (\mathbb{x}', \mathbb{w}')$
2. $semiD' \leftarrow SemiDynark.partial\text{-}\mathcal{K}(\mathbf{pk}, semiD, z_{B_{i-1}}, z, I_{z'-z})$
3. $(\pi', \mathsf{rand}') \leftarrow SemiDynark.\mathcal{U}(\mathbf{pk}, semiD', z, z', \pi, \mathsf{rand})$
4. $\mathbf{state}' \leftarrow SemiDynark.step\text{-}\mathcal{K}(\mathbf{pk}, \mathbf{state}, |I_{z'-z}|)$
5. *Increment* t *by* $|I_{z'-z}|$ *to get* t'*.*
   When $t' = b$*, the updater moves from* B_i *to* B_{i+1} *by setting*
   - $t' \leftarrow 0$
   - $z_{B_{i-1}} \leftarrow z_{B_i}$ *and* $z_{B_i} \leftarrow z'$
   - *Extract* $semiD'$ *from completed* $\mathbf{state}'$
   - *Refresh* $\mathbf{state}'$ *to be* z' *which is the input for the future stepwise preprocessing*
6. *Output* $(\pi', \mathsf{rand}', D')$

a fully dynamic zkSNARK with update time $O(d(\frac{T(\mathcal{K})}{b} + b \cdot T(\mathcal{U})))$. By setting $b = \sqrt{\frac{T(\mathcal{K})}{T(\mathcal{U})}}$, the update time is $O(d\sqrt{T(\mathcal{K})T(\mathcal{U})})$.

5 Evaluations

We have fully implemented our dynamic zkSNARK scheme, DYNARK, and we evaluate its performance in this section.

Software and hardware. DYNARK is implemented in Rust on top of the Groth16 implementation from the arkworks library [ac22a]. Our implementation consists of around 5500 lines of code. The bilinear pairing is instantiated on the BLS12-381 elliptic curve in all of our benchmarks.

All experiments were executed on an AWS c7i.48xlarge machine with 192 vCPUs. Our implementation supports full parallelization utilizing all vCPUs, and we report the performance of both parallelized and serialized implementations in this section. Numbers for setup and preprocessing are averaged over 3 executions, all other numbers are averaged over 10 executions.

5.1 Performance of DYNARK and Comparison with Gorth16

Generation of R1CS. In our experiments, we set the size of the (extended) witness to be the same as the number of R1CS constraints $m = n$. For the

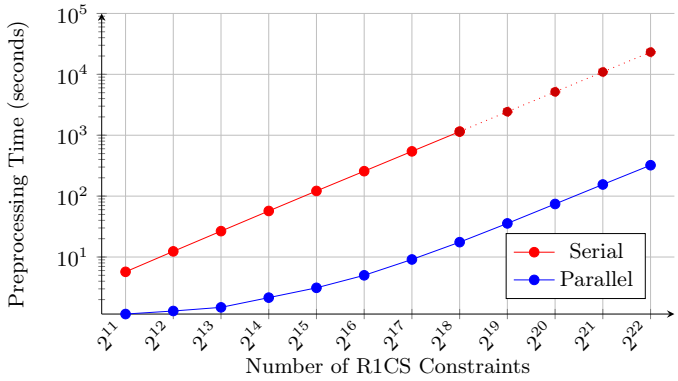


Fig. 1: Preprocessing Time

convenience of measuring the concrete running time, we set $|\Delta_Z| = 2d, |\Delta_A| = |\Delta_B| = d$, and generate random R1CS instances and updates with these constraints[4]. We also provide the calculation of the update time in Table 2 when $|\Delta_Z|, |\Delta_A|, |\Delta_B|$ are not necessarily the same.

Preprocessing time. Dynark has a pre-processing $\mathcal{K}$ on the witness and its running time is presented in Figure 1. As shown in the figure, for $n = 2^{20}$ constraints, it takes 74.3 seconds to compute the preprocessed data with full parallelization. For the serialized version, we only run it up to $n = 2^{18}$ due to the long running time and the dotted line denotes estimation. For $n = 2^{20}$, the serial time is estimated to be 5,158 seconds. This high overhead comes from the $O(n \log n)$ group operations and is consistent with the numbers reported in prior work (e.g.,[DGP+24, Table 3] and the reference implementation of cq [cq-22]). Fortunately, the preprocessing can be significantly accelerated with parallelization using the parallelized DFT and iDFT of group elements implemented in arkworks [ac22b]. We believe the parallel time is reasonable for one-time preprocessing in practice.

Before preprocessing, Dynark also has a trusted setup which generates a few additional elements compared to Groth16 (see $SemiDynark.\mathcal{G}$ in Protocol 2). For $n = 2^{20}$, the trusted setup of Groth16 takes 1.535 seconds, and Dynark is only slightly slower than Groth16 with access to the trapdoor τ. Alternatively, Dynark can take the prover key of any existing Groth16 scheme and compute the additional elements without knowing the trapdoor, which incurs a higher overhead of 59.4 seconds.

Update time. We first give the accurate calculation of the number of group operations per update in Table 2. $|\Delta_Z|, |\Delta_A|, |\Delta_B|$ are the number of updated elements in $z, A \cdot z, B \cdot z$, and b is the number of steps in the fully-dynamic scheme. Table 2 provides a way to precisely calculate the update time when

[4] Setting $|\Delta_Z| = |\Delta_A| = |\Delta_B| = d$ does not allow any meaningful update of the R1CS witness.

Scheme	Group Operation
Semi-Dynamic	$(4\lvert\Delta_Z\rvert + 2\lvert\Delta_A\rvert + 2\lvert\Delta_B\rvert)\mathbb{G}$
Fully-Dynamic	$(4\lvert\Delta_Z\rvert + b + 3\lvert\Delta_A\rvert + 3\lvert\Delta_B\rvert + \frac{n(6+2\log n)}{b}\lvert\Delta_Z\rvert)\mathbb{G}$

Table 2: Number of group operations to update a proof in DYNARK

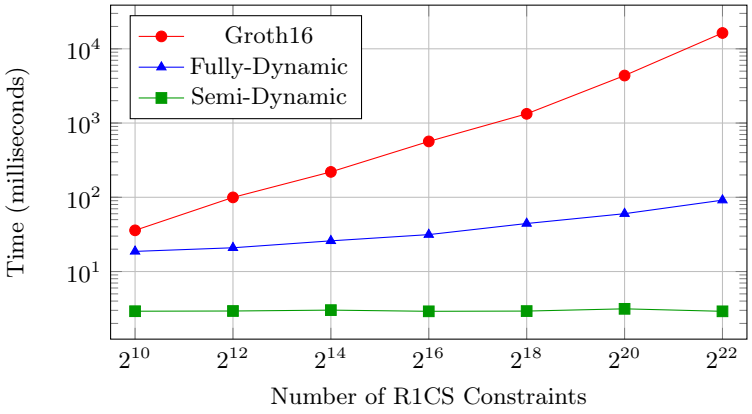


Fig. 2: Update time (parallel) of DYNARK with $d = 1$.

$|\Delta_Z|, |\Delta_A|, |\Delta_B|$ are not necessarily the same in an update. The update time does not depend on $|\Delta_C|$ at all because Δ_C is fully determined by Δ_A, Δ_B to keep the witness valid, and does not appear in the update algorithm at all.

We only report the number of group operations in the table because they are the dominating cost. In our implementation, we use the naive algorithm with $O(d^2)$ field operations to compute the constants of the cross term. This is because for the size of updates we consider in the experiments, the naive algorithm is faster than the sparse convolution with complexity $O(d \log^2 d)$, and neither of them is the dominating cost compared to the group operations.

For the concrete running time, we first fix $d = 1$ and vary the number of R1CS constraints. Figure 2 shows the performance of DYNARK and the comparison with Groth16, which computes new proofs from scratch. As shown in the figure, our update time is significantly faster than Groth16. With parallelization, for $n = 2^{20}$, it merely takes 3 milliseconds to update the proof in our semi-dynamic zkSNARK, and 60 milliseconds in our fully-dynamic zkSNARK. These are 1433× and 73× faster than Groth16, which takes 4.4 seconds to compute the proof from scratch. We also report the complete serial running times in the full version. In serial setting, for $n = 2^{20}$, it takes 3 milliseconds for our semi-dynamic zkSNARK to update the proof, and 1.6 seconds for our fully-dynamic zkSNARK, while it takes 52 seconds for Groth16 to generate the proof. The speedups are 17334× and 33×, respectively. It demonstrates that our speedup does not only come

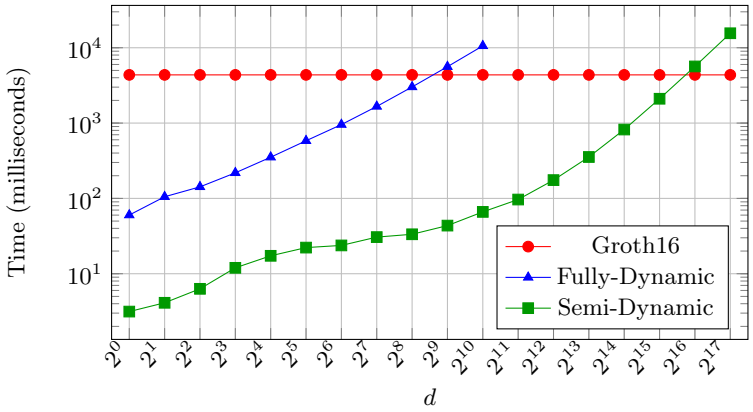


Fig. 3: Update time (parallel) of DYNARK with different d and $n = 2^{20}$.

from the parallelization. In fact, our semi-dynamic zkSNARK is so fast that the parallelization does not help at all.

To better understand the update time of DYNARK with respect to different size of the updates, we fix the size of $n = 2^{20}$ and vary d. Figure 3 shows the performance of DYNARK, with the fixed prover time of Groth16 for $n = 2^{20}$ as a reference. As shown in the figure, our semi-dynamic zkSNARK is extremely fast. With parallelization, it merely takes 2.1 seconds to update a proof with $d = 2^{15}$, which is still more than 2× faster than Groth16. The break-even point is between 2^{15} and 2^{16}. The update time of our fully-dynamic zkSNARK grows linearly with d, which is consistent with our complexity analysis. It takes 3.0 seconds to update the proof with $d = 2^8$, which is 1.4× faster than Groth16. The break-even point is between 2^8 and 2^9. We show similar behavior for the serial time in the full version of the paper, where the break-even points are between 2^{13} and 2^{14} for our semi-dynamic zkSNARK and between 2^6 and 2^7 for our fully-dynamic zkSNARK.

Proof size and verifier time. The proof size of our schemes is 192 bytes and the verifier time is 4.4 milliseconds, which are exactly the same as Groth16.

5.2 Comparison with [WPSP25b]

We further compare the performance of DYNARK with the dynamic zkSNARK scheme in [WPSP25b]. As the implementation of [WPSP25b] is not available, we compare with the numbers reported in an earlier version of the paper [WPSP25a, Table 3], which were obtained on an AWS machine with the same configuration as our machine and with full parallelization.

Table 3 shows the comparison with $d = 1$. Note that DYNARK works for R1CS and the scheme in [WPSP25b] is for Plonkish constraints. R1CS is usually more compact than Plonkish constraints for the same computation. However, we still use the same n to compare with [WPSP25b]. As shown in the table,

Scheme	$\log_2 n$	Update Time(ms)	Verifier Time(ms)	Proof Size
Dynaverse	18	70	95	96 KB
	20	80	125	192 KB
	22	120	179	384 KB
Semi-Dynamic	18	2.9	4.4	192 B
	20	3.1		
	22	3.0		
Fully-Dynamic	18	44.3		
	20	60.1		
	22	91.4		

Table 3: Update time of DYNARK and [WPSP25a], $d = 1$.

the major advantage of DYNARK is that the proof size is significantly smaller than [WPSP25b]. It is merely 192 bytes regardless of n. This is 5,000–20,000× smaller than [WPSP25b]. The verifier time is also faster than [WPSP25b] for all n. In terms of the update time, our semi-dynamic zkSNARK is 23–40× faster than [WPSP25b], while our fully-dynamic zkSNARK is roughly 1.3× faster than [WPSP25b].

Acknowledgments

We thank the authors of [WPSP25b] for helpful discussions on dynamic zkSNARKs. We thank Zhongtang Luo for pointing out the dynamic algorithms and lower bounds for convolutions in [FHM01]. This work is supported by the National Science Foundation (NSF) under Grant No. 2401481 and a Google Research Scholar Award. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of these institutes.

References

ac22a. arkworks contributors. ark-groth16. https://github.com/arkworks-rs/groth16, 2022.

ac22b. arkworks contributors. **arkworks** zksnark ecosystem. https://arkworks.rs, 2022.

AHIV17. Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramaniam. Ligero: Lightweight sublinear arguments without a trusted setup. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 2087–2104, 2017.

APKP24. Kasra Abbaszadeh, Christodoulos Pappas, Jonathan Katz, and Dimitrios Papadopoulos. Zero-knowledge proofs of training for deep neural networks. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 4316–4330, 2024.

BBB+18. Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *2018 IEEE symposium on security and privacy (SP)*, pages 315–334. IEEE, 2018.

BC24. Dan Boneh and Binyi Chen. Latticefold: A lattice-based folding scheme and its applications to succinct proof systems. *Cryptology ePrint Archive*, 2024.

BFK+24. Alexander R Block, Zhiyong Fang, Jonathan Katz, Justin Thaler, Hendrik Waldner, and Yupeng Zhang. Field-agnostic snarks from expand-accumulate codes. In *Annual International Cryptology Conference*, pages 276–307. Springer, 2024.

BFS20. Benedikt Bünz, Ben Fisch, and Alan Szepieniec. Transparent SNARKs from DARK compilers. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 677–706. Springer, 2020.

BMM+21. Benedikt Bünz, Mary Maller, Pratyush Mishra, Nirvan Tyagi, and Psi Vesely. Proofs for inner pairing products and applications. In *Advances in Cryptology – ASIACRYPT 2021*, pages 65–97, 2021.

BSBHR18. Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast Reed-Solomon interactive oracle proofs of proximity. In *45th international colloquium on automata, languages, and programming (icalp 2018)*, 2018.

BSCR+19. Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, and Nicholas P. Ward. Aurora: Transparent succinct arguments for R1CS. In *Advances in Cryptology – EUROCRYPT 2019*, pages 103–128, 2019.

CBBZ22. Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. Hyperplonk: Plonk with linear-time prover and high-degree custom gates. *Cryptology ePrint Archive*, 2022.

CHM+20. Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Noah Vesely, and Nicholas Ward. Marlin: Preprocessing zkSNARKs with universal and updatable SRS. In *Advances in Cryptology – EUROCRYPT 2020*, pages 738–768, 2020.

cq-22. cq reference implementation. https://github.com/geometryxyz/cq, 2022.

DCB25. Trisha Datta, Binyi Chen, and Dan Boneh. Veritas: Verifying image transformations at scale. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 4606–4623. IEEE, 2025.

DGP+24. Moumita Dutta, Chaya Ganesh, Sikhar Patranabis, Shubh Prakash, and Nitin Singh. Batching-efficient ram using updatable lookup arguments. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 4077–4091, 2024.

EFG22. Liam Eagen, Dario Fiore, and Ariel Gabizon. cq: Cached quotients for fast lookups. Cryptology ePrint Archive, Paper 2022/1763, 2022.

FHM01. Gudmund Skovbjerg Frandsen, Johan P Hansen, and Peter Bro Miltersen. Lower bounds for dynamic algebraic problems. *Information and Computation*, 171(2):333–349, 2001.

FK23. Dankrad Feist and Dmitry Khovratovich. Fast amortized kzg proofs. *Cryptology ePrint Archive*, 2023.

FS87. Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Advances in Cryptology — CRYPTO' 86*, 1987.

GAZ+22. Paul Grubbs, Arasu Arun, Ye Zhang, Joseph Bonneau, and Michael Walfish. {Zero-Knowledge} middleboxes. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 4255–4272, 2022.

GGJ+23. Sanjam Garg, Aarushi Goel, Somesh Jha, Saeed Mahloujifar, Mohammad Mahmoody, Guru-Vamsi Policharla, and Mingyuan Wang. Experimenting with zero-knowledge proofs of training. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 1880–1894, 2023.

GGK+25. Sanjam Garg, Aarushi Goel, Dimitris Kolonelos, Sina Shiehian, and Rohit Sinha. Split prover zero-knowledge snarks. In *Public-Key Cryptography – PKC 2025*, pages 353–380, 2025.

GGS87. Apostolos Gerasoulis, Michael D Grigoriadis, and Liping Sun. A fast algorithm for trummer's problem. *SIAM journal on Scientific and Statistical Computing*, 8(1):s135–s138, 1987.

GLS+23. Alexander Golovnev, Jonathan Lee, Srinath Setty, Justin Thaler, and Riad S Wahby. Brakedown: Linear-time and field-agnostic snarks for R1CS. In *Annual International Cryptology Conference*, pages 193–226. Springer, 2023.

Gro16. Jens Groth. On the size of pairing-based non-interactive arguments. In *Advances in Cryptology – EUROCRYPT 2016*, pages 305–326, 2016.

GWC19. Ariel Gabizon, Zachary J Williamson, and Oana Ciobotaru. Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. *Cryptology ePrint Archive*, 2019.

Kil92. Joe Kilian. A note on efficient zero-knowledge proofs and arguments (extended abstract). In *Proceedings of the ACM Symposium on Theory of Computing*, 1992.

LJGK25. Zhongtang Luo, Yanxue Jia, Alejandra Victoria Ospina Gracia, and Aniket Kate. Cauchyproofs: Batch-updatable vector commitment with easy aggregation and application to stateless blockchains. pages 1947–1963, 2025.

LXZ21. Tianyi Liu, Xiang Xie, and Yupeng Zhang. Zkcnn: Zero knowledge proofs for convolutional neural network predictions and accuracy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 2968–2985, 2021.

Mer87. Ralph C Merkle. A digital signature based on a conventional encryption function. In *Conference on the theory and application of cryptographic techniques*, 1987.

Mic00. Silvio Micali. Computationally sound proofs. *SIAM J. Comput.*, 2000.

NT16. Assa Naveh and Eran Tromer. Photoproof: Cryptographic image authentication for any set of permissible transformations. In *2016 IEEE Symposium on Security and Privacy (SP)*, pages 255–271. IEEE, 2016.

PHGR13. Bryan Parno, Jon Howell, Craig Gentry, and Mariana Raykova. Pinocchio: Nearly practical verifiable computation. pages 238–252, 2013.

QSL+25. Wenjie Qu, Yijun Sun, Xuanming Liu, Tao Lu, Yanpei Guo, Kai Chen, and Jiaheng Zhang. zkgpt: An efficient non-interactive zero-knowledge proof framework for llm inference. In *34st USENIX Security Symposium (USENIX Security 25)*, 2025.

RIS23. RISC Zero. Scalable, Transparent Arguments of RISC-V Integrity. Technical report, RISC Zero, 2023.

RT97. John H Reif and Stephen R Tate. On dynamic algorithms for algebraic problems. *Journal of Algorithms*, 22(2):347–371, 1997.

SCG+14. Eli Ben Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. Zerocash: Decentralized anonymous payments from bitcoin. In *2014 IEEE symposium on security and privacy*, pages 459–474. IEEE, 2014.

Set20. Srinath Setty. Spartan: Efficient and general-purpose zkSNARKs without trusted setup. In *Advances in Cryptology – CRYPTO 2020*, pages 704–737, 2020.

SLZ24. Haochen Sun, Jason Li, and Hongyang Zhang. zkllm: Zero knowledge proofs for large language models. *arXiv preprint arXiv:2404.16109*, 2024.

TAB+20. Alin Tomescu, Ittai Abraham, Vitalik Buterin, Justin Drake, Dankrad Feist, and Dmitry Khovratovich. Aggregatable subvector commitments for stateless cryptocurrencies. In *Security and Cryptography for Networks*, 2020.

Tha22. Justin Thaler. Proofs, arguments, and zero-knowledge. *Foundations and Trends® in Privacy and Security*, 4(2–4):117–660, 2022.

vit. An Incomplete Guide to Rollups. https://vitalik.ca/general/2021/01/05/rollup.html.

WPSP25a. Weijie Wang, Charalampos Papamanthou, Shravan Srinivasan, and Dimitrios Papadopoulos. Dynamic zk-SNARKs. https://eprint.iacr.org/archive/2024/1566/20250214:152438, Version 2025-02-14, 2025.

WPSP25b. Weijie Wang, Charalampos Papamanthou, Shravan Srinivasan, and Dimitrios Papadopoulos. Dynamic zk-SNARKs. https://eprint.iacr.org/archive/2024/1566/20250716:152311, Version 2025-07-16, 2025.

WPSP25c. Weijie Wang, Charalampos Papamanthou, Shravan Srinivasan, and Dimitrios Papadopoulos. Dynamic zk-SNARKs. https://eprint.iacr.org/archive/2024/1566/20251013:145022, Version 2025-10-13, 2025.

WTS+18. Riad S Wahby, Ioanna Tzialla, Abhi Shelat, Justin Thaler, and Michael Walfish. Doubly-efficient zkSNARKs without trusted setup. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 926–943, 2018.

WYX+21. Chenkai Weng, Kang Yang, Xiang Xie, Jonathan Katz, and Xiao Wang. Mystique: Efficient conversions for {Zero-Knowledge} proofs with applications to machine learning. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 501–518, 2021.

XZC+22. Tiancheng Xie, Jiaheng Zhang, Zerui Cheng, Fan Zhang, Yupeng Zhang, Yongzheng Jia, Dan Boneh, and Dawn Song. zkbridge: Trustless cross-chain bridges made practical. pages 3003–3017, 2022.

XZS22. Tiancheng Xie, Yupeng Zhang, and Dawn Song. Orion: Zero knowledge proof with linear prover time. In *Annual International Cryptology Conference*, pages 299–328. Springer, 2022.

XZZ+19. Tiacheng Xie, Jiaheng Zhang, Yupeng Zhang, Charalampos Papamanthou, and Dawn Song. Libra: Succinct zero-knowledge proofs with optimal prover computation. In *Annual International Cryptology Conference*, pages 733–764. Springer, 2019.

ZCF24. Hadas Zeilberger, Binyi Chen, and Ben Fisch. Basefold: efficient field-agnostic polynomial commitment schemes from foldable codes. In *Annual International Cryptology Conference*, pages 138–169. Springer, 2024.

ZFZS20. Jiaheng Zhang, Zhiyong Fang, Yupeng Zhang, and Dawn Song. Zero knowledge proofs for decision tree predictions and accuracy. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pages 2039–2053, 2020.